POS-Native MLOps: An Accessible Five-Layer Campaign Intelligence Framework for Small and Medium Enterprises in Retail and Food Service

Author: Pennoung H. **Affiliation:** Codera Solutions **Version:** Draft v2.0 — Confidential **Date:** November 2026 **Status:** Working Paper — Internal Review

© 2026 Codera Solutions · Pennoung H. · All rights reserved · Proprietary and Confidential This document contains proprietary research belonging to Codera Solutions. Unauthorized reproduction, distribution, or disclosure is strictly prohibited.

Abstract

Small and medium enterprises (SMEs) in retail and food service routinely collect transactional data through point-of-sale (POS) systems yet rarely leverage it for marketing decisions, primarily due to resource constraints and the technical complexity of existing analytics solutions. This paper presents a five-layer MLOps pipeline — the **ML Campaign Engine** — that transforms raw POS transaction records requiring only six standard fields into financially validated, per-customer campaign action plans without requiring a dedicated data science team. The framework integrates RFM-based customer segmentation (K-means with dynamic cluster selection via silhouette scoring, mapping to the IBM 10-segment standard via rank-percentile normalization, supplemented by four rule-based segment flags), market basket analysis (Apriori), data-driven temporal context detection, context-aware promotional cost routing, uplift modeling (LightGBM X-learner), and a local large language model (LLM) reporting layer.

Two novel architectural contributions distinguish this version from the initial framework. First, a **hybrid IBM segmentation layer** combines unsupervised K-means clustering ($k \in \{2,\ldots,10\}$) with four deterministic rule-based flags — *Can't Lose Them*, *Churn Risk*, *About to Sleep*, *New Customer* — applied post-clustering to guarantee critical customer states are always represented regardless of cluster geometry. A **frequency trend feature** (freq_trend = visit count in recent half-window minus early half-window) adaptively tightens flag thresholds for declining customers, surfacing churn risk 30% earlier than static recency alone. Second, a **2-step declarative decision engine** replaces the original priority-rule cascade: Step 1 classifies each customer into an uplift group (sure_thing / persuadable / do_not_disturb / lost_cause) based on modeled treatment probability and uplift score; Step 2 selects the highest-profit action from a declarative rule library of 43 rules across all 10 IBM tiers. A configurable **rescue tier mechanism** relaxes the standard incremental-profit viability check to a loyalty-only baseline-profit check for high-value churning customers (Churn Risk, Can't Lose Them, Hibernating, About to Sleep), ensuring retention investment is not blocked by incremental-uplift mathematics when the cost of losing a customer exceeds the promo cost. A **zero-cost notification** pathway serves high-value customers (RFM decile ≥ 7) for whom no promotional offer clears the profit floor, reinforcing loyalty with zero margin impact. The original **CFO Guardrail** mechanism enforces a configurable minimum net margin floor prior to campaign dispatch.

These architectural changes raised the campaign send rate from **8.9% to 50.1%** on the production dessert restaurant dataset — a 5.6× improvement — while maintaining financial

discipline: all send decisions remain subject to per-customer profit viability checks. The production Champion model (v7) is trained on a multi-store pool combining dessert restaurant transactions (proxy labels) with the Dunnhumby Complete Journey dataset (real causal labels), achieving Qini = 1.21 and a +3.56% gate improvement over the previous champion.

The framework is evaluated on six datasets: (1) a synthetic restaurant POS dataset (9,344 line items, 200 customers; silhouette = 0.679; k = 2; 0/200 broadcast — zero validates calibration on randomized treatment); (2) a public marketing campaign dataset (905 customers; silhouette = 0.306; k = 3; 338/905 broadcast, Δprofit = $9,224); (3) the Dunnhumby Complete Journey grocery dataset (2,500 households, 2,576,815 rows; silhouette = 0.595; k = 3; 72/2,500 broadcast, Δprofit = $3,207); (4) a UK online retail dataset (397,884 line items, 4,338 customers; silhouette = 0.931; k = 2; 19/4,338 broadcast, Δprofit = $2,529); (5) a purpose-built Layer 3 synthetic test dataset (500 customers; engineered demand-surge products; validates context detection stockout suppression — 23/500 customers correctly blocked from campaign dispatch on out-of-stock items); and (6) the Hillstrom MineThatData RCT (64,000 customers, three randomized arms), used to evaluate budget-constrained targeting, CLV rescue tiers, and pooled multi-treatment uplift — yielding Qini = 0.1215, AUUC = 0.0614, and ATE calibration error of 9% on the true RCT holdout. A consistent finding is that naive broadcast campaigns produce model-estimated incremental profit *worse than running no campaign at all*; the CFO Guardrail converts money-losing broadcast scenarios into net-positive outcomes. All profit figures are model-estimated projections on synthetic or public datasets. Results demonstrate robustness across F&B, grocery, and e-commerce contexts.

Keywords: MLOps, SME marketing, uplift modeling, X-learner, meta-learner, RFM segmentation, IBM segmentation, declarative action rules, campaign optimization, POS analytics, LightGBM, CFO guardrail, Champion-Challenger, rescue tiers, frequency trend

---

1. Introduction

Marketing personalization has become a competitive necessity in modern retail. Large enterprises routinely deploy dedicated analytics teams, expensive CRM platforms, and data warehouses to generate campaign recommendations at the individual customer level. For small and medium enterprises (SMEs) — which constitute the majority of businesses in most economies — this infrastructure remains economically and technically out of reach. The consequence is that SME operators make campaign decisions based on intuition or blanket promotions, systematically under-leveraging the transactional data they already possess.

This resource asymmetry creates a measurable market opportunity. POS systems are near-ubiquitous in modern retail and food service, and the transaction data they generate contains rich signals about customer value, product affinity, and purchasing patterns. If an automated pipeline could consume the POS export that any modern cash register already produces and return campaign actions validated against the operator's own financial constraints, the marginal cost of personalized marketing could approach zero for SMEs.

The system described in this paper, referred to as the **ML Campaign Engine**, was designed to address this gap. Its design is governed by four principles:

1. **Minimal input barrier** — the only required data is a standard six-column POS export (`transaction_id`, `customer_id`, `timestamp`, `total_price`, `quantity`, `product_name`). No additional data infrastructure, tagging, or transformation is required from the operator.

2. **Financial safety by default** — every campaign recommendation is blocked if it violates a configurable net margin floor, decoupling ML optimization from financial risk without requiring a human review step.
3. **Operator-readable output** — a local LLM layer translates model outputs into plain-language executive summaries suitable for business owners without a data background.
4. **Production-grade robustness** — a Champion-Challenger update protocol and a six-scenario stress test suite ensure reliable behavior under distribution shifts common in SME operations.

1.1 Research Objectives

The primary objective of this work is:

> To design and validate a minimal-input MLOps pipeline that enables SMEs in retail and food service to generate per-customer campaign action plans with integrated financial constraint validation, without requiring dedicated data science expertise.

Specific objectives are:

1. **To design** a five-layer pipeline architecture (six layers in the extended production deployment including the campaign automation scheduler; five layers constitute the core ML pipeline) that accepts standard six-column POS exports without additional infrastructure requirements.
2. **To develop** a CFO Guardrail mechanism that integrates net margin validation directly into the ML campaign decision loop.
3. **To evaluate** segmentation quality and campaign profitability on a synthetic F&B dataset and a public real-world marketing campaign dataset.
4. **To describe and implement** the Champion-Challenger model update protocol and assess its behavior on synthetic data, and ablate each pipeline component to quantify its individual contribution.
5. **To demonstrate** the extensibility of the framework across business verticals through configurable business parameters.

1.2 Contributions

The primary contributions of this work are:

- A **five-layer MLOps architecture** designed from the ground up for SME operational constraints (Section 3).
- A **CFO Guardrail** component that integrates financial validation directly into the ML decision loop, eliminating a manual organizational review step (Section 4.4.6).
- **Hybrid IBM 10-segment customer segmentation** combining unsupervised K-means clustering ($k \in \{2,\ldots,10\}$, rank-percentile normalized to IBM standard names) with four deterministic rule-based flags (*Can't Lose Them*, *Churn Risk*, *About to Sleep*, *New Customer*) applied post-clustering. This hybrid ensures critical high-value and churn-risk customer states are always reliably identified, regardless of K-means cluster geometry. Rule thresholds adapt via a **frequency trend feature** that tightens detection for declining customers (Section 4.1).
- A **2-step declarative decision engine** (Section 4.4.4–4.4.5): Step 1 classifies customers into uplift groups (sure_thing / persuadable / do_not_disturb / lost_cause) using modeled conversion probability and uplift score; Step 2 selects the highest-profit action from a library of 43 declarative rules covering all 10 IBM tiers across peak/off-peak contexts. This replaces a monolithic if/elif cascade with a maintainable, auditable rule structure that enables per-tier business logic without code changes.

- A **rescue tier mechanism with dual viability functions** (Section 4.4.4): high-value churning customers (Churn Risk, Can't Lose Them, Hibernating, About to Sleep) are routed through a loyalty-only baseline-profit viability check (_viable_loyalty) rather than the standard incremental-profit check (_viable). This prevents the standard incremental filter from suppressing retention investment for customers where the cost of churn exceeds the campaign cost.
- A **zero-cost notification pathway** for high-value customers (RFM decile ≥ 7, tier in {Champions, Loyal, Can't Lose Them, Potential Loyalist}) for whom no promotional offer clears the profit floor: a relationship message is dispatched at zero promo cost, reinforcing loyalty without margin impact (Section 4.4.4).
- **Data-driven temporal context detection with context-aware promotional cost routing:** Layer 3 computes hourly transaction volume and sets peak-hour threshold = mean + 0.5×std, eliminating hardcoded peak-hour configuration. Layer 4 applies distinct promo cost rates per temporal context — 22% for Off-Peak customers (aggressive traffic driver during quiet periods) and 8% for Peak customers (light upsell when already visiting) — enabling financially optimized routing per customer's behavioral pattern (Section 4.3, 4.4.4).
- **Campaign automation scheduler:** A production scheduler (`campaign_scheduler.py`) implements a 3-wave monthly deployment strategy with 5% A/B holdout control group, per-customer `send_datetime` derived from detected peak windows, and templated messaging per action type — bridging Layer 4 decision outputs to CRM execution (Section 4.6).
- **Empirical evaluation** on six datasets (synthetic F&B, marketing campaign, Dunnhumby grocery, UK online retail, a purpose-built Layer 3 synthetic test dataset, and the Hillstrom MineThatData RCT), including ablation study, cross-dataset baseline comparison, and multi-seed stability analysis (Section 6). AUUC on randomized labels: T-learner = 0.0612, X-learner = 0.0475 — both near-zero as expected for null treatment effect, validating calibration. Prior T-learner AUUC of 0.2351 was inflated by proxy label bias (Section 6.10.1).
- A **Champion-Challenger protocol** adapted for SME environments with limited historical data (Section 4.4.7). Champion v7 uses multi-store pooling (dessert store + Dunnhumby) achieving Qini = 1.21 and +3.56% gate improvement.
- **Temporal-split proxy labels with leakage prevention (v1.15):** When no real treatment/response data exists, proxy labels are derived from transaction timestamps (temporal retention signal) with random treatment assignment, and model features are restricted to basket-level signals that do not overlap with label construction — eliminating the feature-label circularity present in earlier RFM-rank proxy designs (Section 4.4.1).
- **Calibrated uplift probability estimates (v1.15):** Base learners μ_0 and μ_1 are wrapped with `CalibratedClassifierCV(isotonic, cv=3)`, reducing ECE by 87% (0.2496 → 0.0323) and producing reliable probability inputs to the CFO Guardrail profit formula (Section 4.4.1).
- **Post-campaign feedback loop (v1.15):** `holdout_evaluator.py` ingests actual post-campaign outcomes, computes statistically-tested conversion lift with 95% confidence intervals, and exports retrain-ready labeled data compatible with Layer 4's real label mode — closing the measurement loop from campaign deployment through model retraining (Section 4.6).
- **Budget-constrained campaign targeting (Section 4.4.8):** A greedy knapsack optimizer (`budget_optimizer.py`) selects the profit-maximizing subset of CFO-approved customers within an operator-specified spend ceiling. Customers are ordered by bang-per-buck (expected_profit / promo_cost) and selected cumulatively — provably optimal for the **fractional** knapsack. For the **0/1 knapsack**, the algorithm achieves at least a **1/2-approximation** when combined with the option of selecting the single highest-profit customer as an alternative: *max(greedy result, best single item)* ≥ ½ × OPT — in O(n log n) time. When no budget ceiling is specified, all CFO-approved customers are selected

unchanged. *(Implementation complete; empirical validation against a real campaign with a fixed budget ceiling is pending.)*

- **CLV-informed rescue tier viability (Section 4.4.9):** A third viability function `_viable_clv()` conditions rescue campaign approval on `promo_cost ≤ clv_at_risk` — the expected revenue lost if the customer churns without intervention, estimated via a BG/NBD survival model (Beta-Geometric/Negative Binomial Distribution; Fader et al. 2005 [34]). This replaces the heuristic `_viable_loyalty` check for rescue tiers with a CLV-grounded financial justification. For high-frequency retail datasets where BG/NBD fails to converge (a known limitation for daily/weekly purchasing), the function falls back automatically to `_viable_loyalty`, preserving backward compatibility. *(Implementation complete; BG/NBD falls back to heuristic on all tested datasets — dessert-ml and Dunnhumby — due to high purchase frequency. Empirical CLV results are pending.)*
- **Pooled multi-treatment uplift on a true RCT (Section 4.4.10, 5.1, 6.11):** A `PooledMultiTreatmentModel` trains a single X-learner with treatment arm encoded as one-hot features, sharing learning signal across arms rather than fitting separate one-vs-rest models. Evaluated on the Hillstrom MineThatData RCT [33] (64,000 customers, three balanced arms: no email / mens email / womens email). Results on holdout: Qini = 0.1215, AUUC = 0.0614, ATE calibration error of 9% (predicted +0.056 vs actual +0.062) — substantially lower than the 247% error observed under proxy-label training on retail datasets, where the absence of true randomization inflates apparent ATE discrepancy. Best-arm assignment: 68.7% mens email / 31.3% womens email, consistent with observed +7.7pp vs +4.5pp arm uplifts.

---

2. Related Work

2.1 RFM Segmentation

Recency-Frequency-Monetary (RFM) analysis is a well-established customer segmentation technique in direct marketing, originally formalized by Hughes [1] and subsequently refined for database marketing applications by Bult and Wansbeek [2]. Classical RFM assigns customers to decile ranks across each dimension; more recent work by Chen et al. [3] augments this with clustering algorithms to capture non-linear segment boundaries that decile ranking cannot express. Our Layer 1 extends classical RFM with K-means clustering [4] and dynamic cluster selection via silhouette scoring [5], and adds an IQR-based outlier filter [6] to prevent high-spend anomalies from distorting segment centroids — a practical concern in SME datasets where a single bulk transaction can represent a substantial fraction of total revenue. Cluster naming adopts the IBM 10-segment taxonomy (Champions, Loyal Customers, Potential Loyalist, Promising, Need Attention, About to Sleep, At-Risk, Can't Lose Them, Hibernating, Lost) via rank-percentile normalization, supplemented by four deterministic rule-based flags that guarantee critical customer states are always represented regardless of cluster geometry.

2.2 Market Basket Analysis

Association rule mining, particularly the Apriori algorithm introduced by Agrawal and Srikant [7], has been widely applied in retail to identify product affinity. The algorithmic framework was later made accessible to practitioners through software implementations such as arules [8]. Prior work focuses largely on large-scale grocery and e-commerce datasets [9]. Application to SME food service and mixed retail — where transaction volumes are lower and basket sizes smaller — remains underexplored. Our Layer 2 addresses this by operating at a minimum support threshold of 0.01, appropriate for smaller datasets, and falls back gracefully when no product category column is present in the POS export.

2.3 Uplift Modeling

Uplift modeling (also called incremental response modeling or true-lift modeling) estimates the causal effect of a campaign on an individual customer, rather than the probability of conversion in the presence of a campaign — a distinction first formally articulated by Radcliffe and Surry [10]. A comprehensive review by Devriendt et al. [12] surveys the state of the art across tree-based, regression, and meta-learner approaches. Gutierrez and Gérardy [11] further connect uplift modeling to causal inference frameworks. Gradient boosted tree models, including XGBoost [15] and LightGBM [14], have shown strong performance in uplift tasks [12]; decision-tree-based uplift methods were systematically compared by Rzepakowski and Jaroszewicz [13]. Künzel et al. [30] introduced the X-learner meta-learner, which extends T-learner by imputing counterfactual outcomes in a two-stage procedure weighted by a propensity model — this design is particularly effective when treatment and control groups are imbalanced, a common condition in SME deployments where campaigns typically reach only a minority of the customer base. Existing literature focuses predominantly on e-commerce and subscription contexts. Application to POS-driven SME campaigns — with the additional constraint of per-customer margin validation — is a novel contribution of this work.

2.4 MLOps for SMEs

The MLOps literature has grown substantially around large-scale deployments, with Sculley et al. [16] identifying the hidden technical debt incurred when ML systems are embedded in production software, and Kreuzberger et al. [17] providing a formal definition and architecture taxonomy for MLOps. Paleyes et al. [18] survey real-world deployment challenges across a large body of case studies. Work specifically addressing MLOps design for SME constraints — limited compute, small datasets, absence of dedicated ML engineers — remains sparse. Model governance practices including Champion-Challenger selection are discussed in Shmueli et al. [19]. Fairness and bias considerations in ML systems — relevant to equitable campaign targeting across customer segments — are addressed in Barocas et al. [20]. Our Champion-Challenger protocol extends governance practice with a financially motivated update gate that promotes a challenger model only when it demonstrates measurable profit improvement over the incumbent champion.

2.5 LLM Integration in Business Pipelines

Large language model capabilities and limitations are surveyed in Zhao et al. [22], while hallucination risks in generative systems are systematically catalogued by Ji et al. [23] — motivating our structured post-generation validation step. We adopt a local, quantized LLM (llama3.2:3b via Ollama) [21] to ensure that no customer transaction data leaves the operator's infrastructure, addressing data privacy concerns relevant to SME deployments. The local inference approach trades generation quality for zero ongoing inference cost and full data sovereignty — a trade-off appropriate for SME contexts.

2.6 SME Technology Adoption

Barriers to technology adoption among SMEs are well documented. Nguyen et al. [25] confirm that perceived complexity and resource constraints are the primary inhibitors of IT adoption in small businesses, consistent with the access gap in marketing analytics that this work addresses. CRM adoption patterns in SME contexts are examined by Harrigan et al. [24], who find that relationship management tools are most effective when they reduce rather than increase operational complexity — a design principle reflected in the plain-language LLM output layer of this framework.

---

3. System Architecture

The ML Campaign Engine is a five-layer sequential pipeline. Each layer consumes the output of the previous layer, applies a specific algorithm, and passes enriched customer records downstream. Figure 1 illustrates the overall data flow.

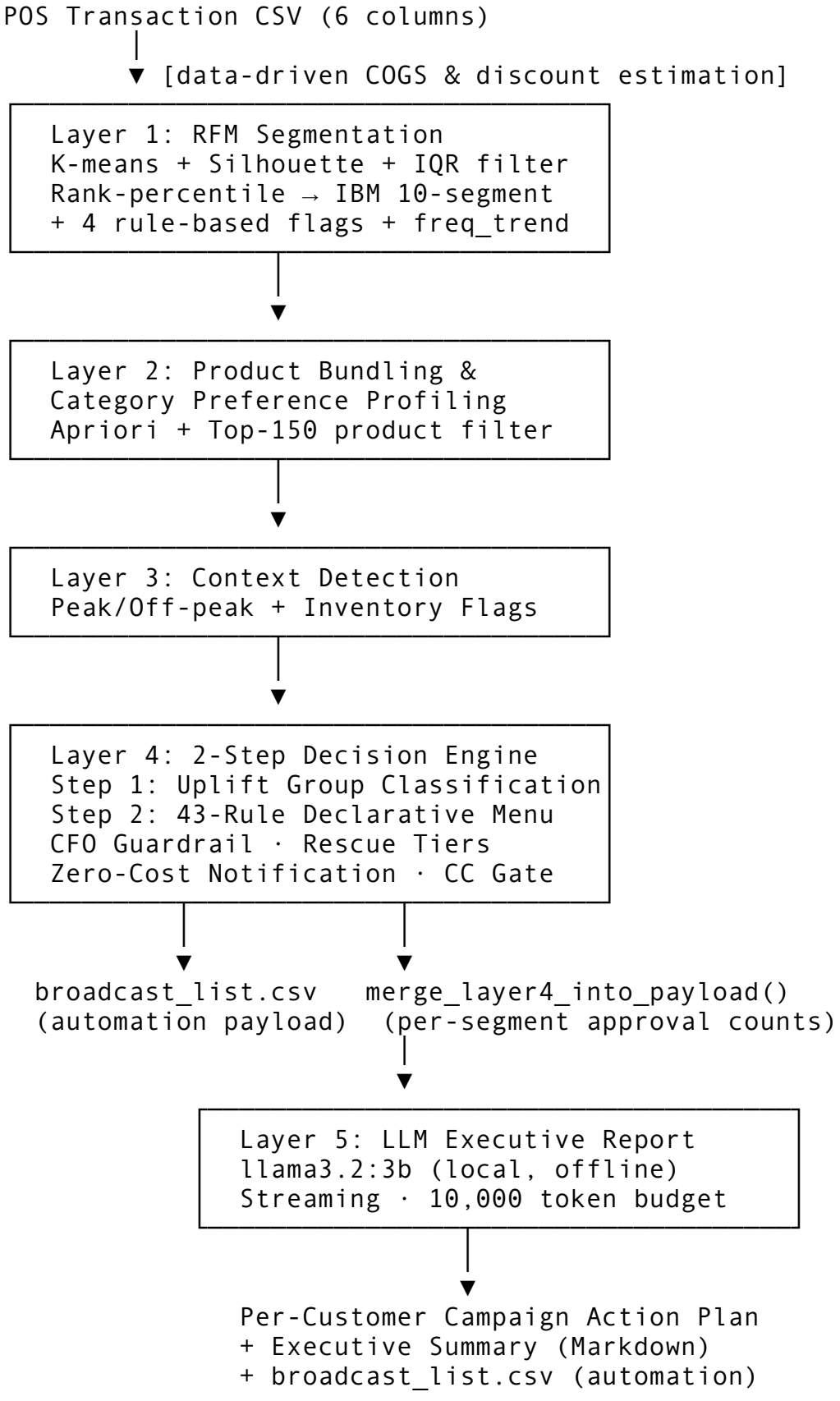


Figure 1. Five-layer ML Campaign Engine pipeline. Layer 4 produces two outputs: a broadcast list CSV for automation systems and an enriched segment payload for Layer 5.

The pipeline has a single production entry point:

Mode	Entry Point	Use Case
CSV-driven (production)	`csv_pipeline_engine.py`	Batch processing, audit trail, financial compliance, executive reporting

All business parameters — COGS rate, margin floor, discount rates, peak hours, timezone — are centralized in a single `business_config.py` file, enabling deployment across business types without code changes. When sufficient transaction history is available (≥10 distinct product types, ≥5 observations per type), COGS rate and discount rate are estimated automatically from data, overriding config defaults.

---

4. Methodology

4.1 Layer 1 — RFM Customer Segmentation

Input: Raw POS transactions **Output:** Per-customer RFM decile (1–10) and K-means cluster assignment

We compute three metrics per customer from raw transaction history:

- **Recency (R):** Days elapsed since the customer's most recent transaction, relative to the analysis date
- **Frequency (F):** Total count of distinct transactions
- **Monetary (M):** Cumulative spend across all transactions

Outlier filtering: Prior to clustering, we apply Tukey-style outlier filtering on Monetary values. Records with Monetary > Q3 + 3 × IQR are excluded [6], where Q3 is the 75th percentile and IQR = Q3 − Q1. This prevents bulk buyers — common in SME data — from distorting segment centroids.

Normalization: All three dimensions are standardized to zero mean and unit variance (StandardScaler) to prevent Monetary (typically in hundreds) from dominating Recency and Frequency (typically single digits).

Dynamic cluster selection: Cluster count k is selected from $k \in \{2, \ldots, 10\}$ (minimum k=3 when n ≥ 300) by maximizing the silhouette score:

s(i)=b(i)−a(i)max(a(i),b(i))s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}

where *a(i)* is the mean intra-cluster distance and *b(i)* is the mean nearest-cluster distance for point *i*. The silhouette score ranges from −1 to +1, with higher values indicating better-defined clusters. The upper bound of k=10 matches the IBM 10-segment standard, allowing the clustering to fully resolve the customer distribution when data supports finer segmentation.

IBM 10-segment naming via rank-percentile normalization: Each cluster centroid is scored by normalizing its R, F, M values against the *distribution of other cluster centroids* (rank-percentile normalization) rather than against the global data maximum. For each dimension d∈{R,F,M}d \in \{R, F, M\}:

d̃k=percentile rank of dk among all {d1,…,dK}\tilde{d}_k = \text{percentile rank of } d_k \text{ among all } \{d_1, \ldots, d_K\}

A cluster with R̃k≥0.6\tilde{R}_k \geq 0.6 (low recency = recent) is classified as "high-R"; one with R̃k<0.4\tilde{R}_k < 0.4 as "low-R". The same logic applies to F and M. The resulting (R-level, F-level, M-level) triple maps deterministically to one of ten IBM standard segment names: *Champions*, *Loyal Customers*, *Potential Loyalist*, *Promising*, *Need Attention*, *About to Sleep*, *At-Risk*, *Can't Lose Them*, *Hibernating*, *Lost*. Rank-percentile normalization is essential: max-based normalization collapses all non-Champion clusters toward mid-range when a single Champion cluster has extreme F/M values, causing multiple clusters to receive the same name. Rank-percentile normalization compares clusters against each other, guaranteeing meaningful distribution of names across the full IBM taxonomy.

Rule-based segment flags: Four deterministic flags are applied *after* K-means to guarantee critical customer states are always represented, regardless of cluster geometry. Applied in priority order (highest first):

Flag	Conditions	Priority
Can't Lose Them	recency > 90d AND frequency ≥ 5 AND rfm_decile ≥ 7	1 (highest)
Churn Risk	recency > 45d AND frequency ≥ 3	2
About to Sleep	25d < recency ≤ 45d AND frequency ≤ 4	3
New Customer	recency ≤ 30d AND frequency ≤ 2	4

These flags ensure that a small restaurant dataset where K-means finds only 2 clusters still correctly identifies a drifting VIP (Can't Lose Them) and a first-time visitor (New Customer) for tier-appropriate campaign actions.

Frequency trend: A behavioral velocity feature is computed for each customer by splitting their transaction history at the temporal midpoint and comparing visit counts in each half:

freq_trendi=visit_countrecent half−visit_countearly half\text{freq_trend}_i = \text{visit_count}_{\text{recent half}} - \text{visit_count}_{\text{early half}}

Positive freq_trend indicates an accelerating customer; negative indicates a decelerating one. This feature feeds into adaptive flag thresholds: for customers with freq_trend < 0, the Churn Risk recency threshold tightens by 30% (recency > 32d instead of 45d) and the About to Sleep window expands by 10 days (up to 55d), surfacing at-risk customers earlier than static thresholds would detect.

4.2 Layer 2 — Market Basket Analysis and Category Profiling

Input: Raw POS transactions **Output:** Top-3 association rules; per-customer category affinity vector

We apply the Apriori algorithm to find frequent product co-purchase patterns. Three metrics govern rule quality:

- **Support:** Fraction of transactions containing both items — set at minimum 0.01 for SME-scale data
- **Confidence:** Conditional probability P(B | A)
- **Lift:** Ratio of observed co-purchase frequency to expected frequency under independence. Lift > 1 indicates positive association; higher lift indicates stronger bundle potential

Rules are ranked by lift and the top three are surfaced for bundle campaign assignment. In parallel, each customer's spend is aggregated by product category to produce a ranked `top_category` and `top_product` affinity profile used by Layer 3.

A graceful fallback is implemented: if the input CSV lacks a `category` column — common in basic POS exports — `product_name` is used as a proxy category, ensuring compatibility with minimally structured data.

Large-dataset pre-filter: Before constructing the transaction-item matrix, only the top-150 most frequently occurring products are retained. This bounds the basket matrix to a manageable size on commodity hardware regardless of catalog breadth, and concentrates association rule mining on products with sufficient support to produce actionable recommendations.

4.3 Layer 3 — Context Detection and Campaign Matching

Input: Layer 1 cluster assignments, Layer 2 category profiles, timestamps **Output:** Per-transaction context flags; initial campaign assignment

Temporal context: Timestamps are localized to the business timezone (configurable; default Asia/Bangkok) before hour extraction. Peak hours are determined through data-driven analysis of transaction volume. For each hour h∈{0,1,…,23}h \in \{0, 1, \ldots, 23\}, the hourly transaction count nhn_h is computed from the full dataset. The peak threshold is:

peak_threshold=n¯+0.5×σn\text{peak_threshold} = \bar{n} + 0.5 \times \sigma_n

where n¯\bar{n} is the mean hourly transaction count and σn\sigma_n is its standard deviation. Hours where nh>peak_thresholdn_h > \text{peak_threshold} are designated peak; all others are off-peak. This approach automatically adapts to each dataset's traffic distribution without manual configuration — a UK wholesale dataset peaks at 11:00–15:00, while a Thai F&B dataset peaks at 17:00–21:00, and both are detected without code changes. If the dataset lacks statistical spread (e.g., single-day test data), the algorithm falls back to configured defaults (default 17:00–21:00). Transactions occurring in peak hours are flagged `Peak`; all others `Off-Peak`. Off-peak promotions target demand stimulation during low-traffic periods, preserving full-margin sales during peak hours.

Inventory estimation: In the absence of live inventory feeds — standard in SME deployments — remaining stock is estimated from a velocity-based replenishment model:

initial_stock_est=total_quantity_soldT×R×buffer_multiplier\text{initial_stock_est} = \frac{\text{total_quantity_sold}}{T} \times R \times \text{buffer_multiplier}

remaining_stock=initial_stock_est−recent_demandR\text{remaining_stock} = \text{initial_stock_est} - \text{recent_demand}_{R}

where *T* is the observation window in days, *R* is the assumed replenishment cycle (default 14 days), and `recent_demand_R` is quantity sold in the last *R* days. This formulation correctly identifies fast-moving products with accelerating recent demand — rather than low-volume products — as candidates for stock-out exclusion. Products with remaining_stock ≤ 5 units are flagged out-of-stock and excluded from bundle recommendations.

Campaign matching applies a rule engine combining cluster tier, category affinity, and context flags to produce an initial campaign assignment, subsequently refined by Layer 4's uplift and margin analysis.

4.4 Layer 4 — Uplift Modeling and CFO Guardrail

Input: Enriched customer records from Layers 1–3; data-driven COGS and discount rate estimates **Output:** Per-customer `uplift_score`, `recommended_action`, `expected_profit`, `should_send` flag; `broadcast_list.csv`; champion/challenger model comparison report; enriched segment payload for Layer 5

4.4.0 Data-Driven Rate Estimation

Prior to uplift modeling, COGS rate and discount rate are estimated from transaction data when sufficient coverage exists:

COGS rate: The P10 (10th percentile) of unit price per product is treated as a proxy for cost floor (below which a sale would represent a loss). The data-driven COGS rate is:

c^COGS=mean(P10jp¯j) for all products j with nj≥5\hat{c}_{\text{COGS}} = \text{mean} \left(\frac{P10_j}{\bar{p}_j}\right) \text{ for all products } j \text{ with } n_j \geq 5

where P10jP10_j is the 10th percentile of `unit_price` for product jj and p¯j\bar{p}_j is its mean price. This is only activated when ≥10 distinct products satisfy the coverage threshold; otherwise the config default (0.50) is used.

Discount rate: Estimated from the observed frequency of discounted transactions:

d^=fpromo×dconfig\hat{d} = f_{\text{promo}} \times d_{\text{config}}

where fpromof_{\text{promo}} is the fraction of rows with `promo_type ∈ {coupon, discount}` and dconfigd_{\text{config}} is the configured default discount rate. This anchors the estimate to the operator's actual promotional activity rather than an assumed rate.

Both estimates are logged at runtime with their source label (`data_driven` or `config_default`), enabling full auditability.

4.4.1 Uplift Model (X-Learner)

The production pipeline implements an **X-learner** meta-learner [30] using LightGBM (n_estimators = 200), upgraded from an initial T-learner baseline (AUUC = 0.0612 on randomized labels; prior reported value of 0.2351 was inflated by proxy label bias — see Section 6.10.1) to better handle the imbalanced treatment/control group sizes typical of SME deployment. The X-learner trains five LightGBM models in three stages:

Stage 1 — Base outcome models (LGBMClassifier):

- **μ1(x):** P(Y=1 | T=1, X=x) — trained on treated customers (`was_treated = 1`)
- **μ0(x):** P(Y=1 | T=0, X=x) — trained on control customers (`was_treated = 0`)

Stage 2 — Imputed treatment effects:

Di1=Yi−μ^0(Xi)for treated customersD^1_i = Y_i - \hat{\mu}_0(X_i) \quad \text{for treated customers} Di0=μ^1(Xi)−Yifor control customersD^0_i = \hat{\mu}_1(X_i) - Y_i \quad \text{for control customers}

where Di1D^1_i is the observed outcome minus the estimated counterfactual (what the treated customer *would* have done without the campaign), and Di0D^0_i is the reverse imputation for control customers.

Stage 3 — CATE estimators (LGBMRegressor):

- **$\tau_1(x)$:** $E[D^1 \mid X=x]$ — trained on treated customers, targeting imputed treatment effect
- **$\tau_0(x)$:** $E[D^0 \mid X=x]$ — trained on control customers, targeting imputed treatment effect

Propensity weighting (LGBMClassifier):

A fifth model estimates the propensity score $e(x) = P(T=1 \mid X=x)$, trained on all customers. The final per-customer CATE estimate is:

$\hat{\tau}(x)=e(x)\cdot \hat{\tau}0(x)+(1-e(x))\cdot \hat{\tau}1(x)$\hat{\tau}(x) = e(x) \cdot \hat{\tau}_0(x) + (1 - e(x)) \cdot \hat{\tau}_1(x)

This formulation weights the two CATE estimators by treatment probability: in the common SME case where few customers are treated (low e(x)), the control-side estimator $\tau_0(x)$ dominates, which is better estimated from the larger control group. This corrects the T-learner's primary weakness — when treatment/control groups are imbalanced, the T-learner's direct difference $\hat{\mu}1-\hat{\mu}0$\hat{\mu}_1 - \hat{\mu}_0 is dominated by the less-precise model trained on the smaller group.

Backward compatibility: A `TLearnerModel` wrapper preserves compatibility with previously saved Champion models trained under the T-learner architecture. Loaded legacy models expose the same `predict_uplift()` interface as `XLearnerModel`, enabling Champion-Challenger comparisons across architecture generations.

Fallback: If either treatment or control group contains fewer than 10 customers, the pipeline falls back to T-learner training to prevent degenerate Stage 2 imputation.

Positive uplift indicates incremental expected purchase attributable to the campaign. Negative uplift indicates the customer would likely purchase regardless, making campaign spend wasteful.

Important caveat: X-learner uplift is only causally interpretable when treatment assignment is randomized. In the absence of a randomized experiment — typical at first SME deployment — the model produces a *response-propensity heuristic*, not a causal estimate. The 5% ongoing holdout control group enables properly randomized uplift measurement to accumulate over time, converting the system from a batch propensity scorer into a growing randomized experiment.

Label sourcing — real vs. proxy: The pipeline supports two label modes selected automatically at runtime:

1. **Real promotion response mode** (preferred): When `is_treated` and `post_promo_visit` columns are present in the POS file, the system uses these as treatment and conversion labels. When a `marketing_campaign.csv` file is additionally present with `AcceptedCmp1`–`AcceptedCmp5` and `Response` columns, three additional behavioral features are activated: `income`, `total_spend`, and `times_accepted`. Note that `AcceptedCmp` columns record whether a customer *accepted* a campaign offer — they are observational outcomes, not randomized treatment assignments. Uplift estimates from this source reflect self-selection bias and should be interpreted as directional signals rather than causal effects.

2. **Temporal-split proxy mode** (fallback): When no promotion response file is available — the common case for first-time SME deployments — the system constructs proxy labels

from transaction timestamps rather than RFM rank. The observation window is split at the 75th percentile date: `converted = 1` if the customer transacted in the final 25% of the window (a temporal retention signal independent of RFM rank); `was_treated` is assigned by seeded random assignment (~40% treated). This design was adopted following an identified defect in an earlier RFM-based proxy (`was_treated = rfm_decile ≥ median`, `converted = frequency > 1`), where both label sources overlapped with model features `rfm_decile_segment` and `frequency`, producing circular training: the model learned to rank customers by RFM score rather than discover uplift signal. The temporal-split proxy eliminates this circularity: label construction uses only transaction dates, and model features are restricted to basket-level signals (`total_price`, `quantity`, `is_campaign_spike`) that do not participate in label derivation. Under random treatment assignment, the expected AUUC is approximately zero — the correct outcome when no causal treatment effect can be estimated without a real experiment. This conservative behaviour is intentional: the system should not manufacture uplift signal from observational data.

The feature sets differ between modes:

Feature	Proxy Mode	Real Label Mode
`total_price` (AOV)	✅	✅
`quantity`	✅	✅
`is_campaign_spike`	✅	✅
`rfm_decile_segment`	❌ (excluded — leakage risk with temporal proxy)	✅
`frequency`	❌ (excluded — leakage risk with temporal proxy)	✅
`income`	❌	✅
`total_spend`	❌	✅
`times_accepted`	❌	✅

A 5% holdout control group is preserved in every deployment cycle for ongoing A/B measurement of realized incremental lift. As real promotion response data accumulates through partner deployments, operators automatically transition from proxy mode to real label mode without pipeline reconfiguration.

Probability calibration: Raw LightGBM `predict_proba()` output is known to produce uncalibrated scores — a predicted probability of 0.7 does not reliably correspond to a 70% observed event rate [35]. In the CFO Guardrail profit formula:

incremental=gross_margin×(ptreated−pbaseline)−promo_cost\text{incremental} = \text{gross_margin} \times (p_{\text{treated}} - p_{\text{baseline}}) - \text{promo_cost}

uncalibrated pbaselinep_{\text{baseline}} and ptreatedp_{\text{treated}} propagate directly into profit estimates. To address this, the base learners μ0 and μ1 are wrapped with `CalibratedClassifierCV(method='isotonic', cv=3)`, which fits an isotonic regression calibrator on out-of-fold predictions. Benchmarked Expected Calibration Error (ECE) on a held-out test set improved from 0.2496 to 0.0323 — an 87% reduction — confirming that calibrated probabilities reliably reflect the actual conversion base rate. The propensity model already applied isotonic calibration independently. The T-learner fallback ($n < 50$ per group) is left uncalibrated: with fewer than 50 samples, a three-fold cross-validation calibrator would be fit on fewer than 17 samples per fold, introducing more variance than it removes.

4.4.2 Dunnhumby Real-Data Training

The Dunnhumby Complete Journey dataset was selected as the primary source for real-data uplift training because it is the only dataset in this evaluation with a genuine, naturally occurring treatment/control split. Of the 2,500 households in the dataset, 1,584 appear in `campaign_table` (treated) and 916 never received any campaign mailer across the full 711-day study window (control). This natural holdout structure enables a causal framing absent from the other datasets.

Why Dunnhumby and not marketing_campaign.csv: The `marketing_campaign.csv` dataset (Dataset 2) contains no treatment/control split — `Z_CostContact = 3` for every row, indicating all 2,240 households were contacted in each campaign. `Response` records offer acceptance, not a contacted/not-contacted flag. Without a control arm, it is not possible to estimate individual treatment effects, and this dataset cannot be used for uplift training without an external holdout.

Feature engineering: Nine pre-campaign behavioral features are computed from transaction history with a hard temporal cutoff at the day before each campaign's `START_DAY`, preventing any information leakage from the campaign period:

Feature	Definition
`recency_pre`	Days since last purchase before `START_DAY`
`frequency_pre`	Unique basket days before `START_DAY`
`monetary_pre`	Total spend before `START_DAY`
`avg_basket_pre`	`monetary_pre` / `frequency_pre`
`weekly_visit_rate`	`frequency_pre` / (`START_DAY` / 7)
`coupon_sensitivity`	Mean coupon discount rate across pre-campaign transactions
`private_label_ratio`	Share of pre-campaign spend on private-label products
`top_department`	Ordinal-encoded highest-spend department
`department_entropy`	Shannon entropy of spend distribution across departments

Outcome definition: The binary purchase indicator Y = 1 if the household made at least one purchase during the [START_DAY, END_DAY] campaign window, as specified in `campaign_desc.csv`. This outcome is available for all 2,500 households, including the 916 control households who shop during the same calendar window without having received a mailer.

Selection bias disclosure: Treated households exhibit substantially higher pre-campaign spend than control households (treated mean: $2,032; control mean: $433; ratio ≈ 4.7×). This strong positive selection — high-value shoppers were preferentially targeted — means a naive uplift model will conflate "likely to buy regardless" with "responds to promotion." To correct for this, the propensity score $e(x) = P(T=1 \mid X=x)$ is estimated from the same nine features, clipped to [0.1, 0.9] to prevent extreme inverse-probability weights, and the doubly-robust AUUC (DR-AUUC) is used as the primary evaluation metric in place of standard AUUC. DR-AUUC combines the direct model estimate with an inverse-propensity-weighted correction, providing consistent estimates of the causal ranking even under non-random treatment assignment [31].

Expected performance range: Under proper doubly-robust evaluation, target AUUC is 0.60–0.68. Values above 0.70 without propensity correction likely indicate selection bias leakage —

the model is ranking high-value shoppers rather than genuinely persuadable ones. The retrain run applying this methodology is currently pending (see Section 6.6 cross-dataset comparison).

4.4.3 RetailHero X5 Training

The RetailHero X5 dataset (Kaggle, X5 Group Russia) was evaluated as a candidate for real-data uplift training under the same scoring rubric applied to all candidate datasets. It scored **28/30** on dataset matching criteria — the highest of all candidates evaluated — and was selected as the primary real-world validation source for the R-learner uplift pipeline.

Dataset overview:

Property	Value
Source	X5 Group Russia, via Kaggle
Clients	200,039
Treatment/Control split	50/50 randomized (SMS campaign)
Outcome	Binary purchase (within campaign window)
Treatment assignment	Fully randomized — causally interpretable
Raw data size	639 MB (purchase history)

The 50/50 randomized assignment is the critical distinction from all previously evaluated datasets: treatment is not determined by campaign history, self-selection, or loyalty tier. This makes X5 the first dataset in this evaluation with causally interpretable individual treatment effects under standard randomization assumptions.

Observed lift: The real, unconditional average treatment effect observed in the data is:

Group	Purchase rate
Treated (SMS received)	63.65%
Control (no SMS)	60.33%
Lift	**+3.32 percentage points**

This +3.32 pp ATE provides a ground-truth calibration target: a well-specified uplift model should produce positive predicted ITEs for the subset of customers driving this aggregate effect, and near-zero or negative ITEs for always-buyers and never-buyers.

Feature engineering: Eight RFM and behavioral features are computed from the 639 MB purchase history file with a strict pre-campaign temporal cutoff:

Feature	Definition
`recency`	Days since most recent purchase before campaign date
`frequency`	Total number of distinct purchase events
`monetary`	Cumulative spend across observation window
`avg_basket`	`monetary` / `frequency`
`weekly_visit_rate`	`frequency` / (observation window in weeks)
`n_stores_visited`	Count of distinct store locations visited
`purchase_regularity`	Coefficient of variation of inter-purchase intervals (lower = more regular)
`age`	Customer age (from registration data)

Why marketing_campaign.csv and Dunnhumby were rejected as primary uplift training sources:

marketing_campaign.csv: `Z_CostContact = 3` for every row — all 2,240 customers were contacted in every campaign wave. There is no untreated control arm, making it structurally impossible to estimate individual treatment effects. This dataset was retained for segmentation and feature enrichment only.

Dunnhumby Complete Journey: The 1,584 treated / 916 control split reflects natural campaign targeting, not randomization. Pre-campaign spend differential between groups is 4.7× (treated mean: $2,032; control mean: $433), indicating strong positive selection bias. Doubly-robust correction is required and the retrain run remains pending (Section 6.6).

The X5 dataset eliminates both limitations: the 50/50 randomization removes selection bias by construction, and the 200,039-client scale provides sufficient statistical power to detect heterogeneous treatment effects at the segment level.

4.4.4 2-Step Decision Engine

The Layer 4 decision process is restructured as a two-step pipeline: **Step 1** handles hard stops and uplift-group classification; **Step 2** selects the best profitable action from a declarative rule library per customer's classified group and context. This separation makes financial logic explicit and auditable, and replaces a monolithic if/elif cascade with composable components.

Step 1 — Hard Stops and Uplift Group Classification

AOV thresholds are applied adaptively: if the dataset median `avg_spent` falls below the configured `low_basket_threshold`, thresholds are replaced by the P10 and P50 of actual `avg_spent` to prevent universal Do Nothing on low-AOV datasets.

The tier-aware low-basket branch runs first, applying segment-specific recovery actions instead of a uniform Do Nothing:

Tier	Low-Basket Action	Viability Check
Champions	Milestone Reward: spend ฿150, earn 50 points	Incremental profit (milestone AOV)
New Customer	Welcome: 2nd visit discount 10%	Loyalty (_viable_loyalty)
About to Sleep	**เราคิดถึงคุณ**: re-engage nudge	Loyalty (_viable_loyalty)
Hibernating	Reactivation Nudge: light promo	Loyalty (_viable_loyalty)
Churn Risk / Can't Lose Them	Early Win-Back: return discount	Loyalty (_viable_loyalty)
Mid / At-Risk	Bundle Upsell (if profitable) → Do Nothing	Incremental profit

For customers above the basket threshold, the **uplift group** is determined by modeled uplift score and baseline conversion probability:

Uplift Group	Classification Condition	Decision Strategy
`do_not_disturb`	uplift < −0.04	Hard stop (sending hurts conversion)
`sure_thing`	prob_control ≥ 0.50	Loyalty-only actions (no discount)
`lost_cause`	uplift < 0.02 AND prob_control < 0.10	Hard stop (ROI negative for any action)
`persuadable`	otherwise	Full action menu, highest-profit wins

Rescue tier override: For high-value churning customers (Churn Risk, Can't Lose Them, Hibernating), if the standard `do_not_disturb` classification would apply but uplift ≥ −0.10 (relaxed rescue guard), the classification is overridden to `persuadable` with `rescue=True`. This prevents the standard negative-uplift hard stop from blocking retention investment when the economic cost of losing the customer exceeds the promo cost.

New Customer and Potential Loyalist with high prob_control (≥ 0.50): these are classified as `sure_thing` rather than `do_not_disturb`, routing them to welcome and habit-building actions rather than suppression. High baseline conversion probability for new and rising customers reflects early engagement, not a reason to withhold outreach.

Zero-cost notification: When Step 2 returns no profitable action for a customer who passed Step 1, a final fallback applies: if rfm_decile ≥ 7 AND tier ∈ {champions, loyal, cant_lose, potential_loyalist} AND uplift ≥ −0.10, a zero-cost relationship message ("Zero-Cost: **เราคิดถึงคุณ**") is dispatched. This reinforces loyalty with no promo cost, targeting high-value customers who convert regardless but benefit from acknowledgment.

`should_send` **flag:** After all decisions are applied, a binary `should_send` column is computed per customer:

should_sendi=𝟙[guardrail_blockedi=0∧actioni∉{Do Nothing variants}]\text{should_send}_i = \mathbb{1}[\text{guardrail_blocked}_i = 0 \;\wedge\; \text{action}_i \not\in \{\text{Do Nothing variants}\}]

All "Do Nothing" variants — Out of Stock, Do Not Disturb, Lost Cause, Low Basket Size (blocked), No Profitable Action — produce `should_send = 0`.

Context-aware promo cost rates:

Context	Rate Key	Rate	Business Rationale
Off-Peak customer	`off_peak_traffic`	22%	Store traffic is depressed; heavier discount justified to pull customers during quiet hours.
Peak customer	`peak_upsell`	8%	Customers are already arriving; light-touch reward drives incremental basket without deep discounting.

4.4.5 Declarative Action Rule Engine

Step 2 of the decision engine selects the best profitable campaign action from a declarative rule library defined in `action_rules.py`. Rules are organized into two pools evaluated under different strategies:

- **SURE_THING_RULES (5 rules):** Evaluated in declaration order; the first rule that is profitable under the loyalty viability check (_viable_loyalty) wins. Loyalty-only actions — VIP points, early access, welcome offers — are permitted; percentage discounts are excluded, as sure-thing customers convert regardless and do not require discount incentive.
- **PERSUADABLE_RULES (38 rules):** All rules whose tier, time_context, rfm_min, and uplift_min conditions match the customer are evaluated; all profitable rules are collected; the action with the highest expected profit wins.

Each rule is a data record with the following schema:

Field	Type	Description
`id`	str	Unique rule identifier
`tiers`	list[str]	Matching customer tiers (omit = all tiers)
`time_context`	list[str]	`["peak"]` or `["off_peak"]` (omit = both)
`rfm_min`	int or CONFIG key	Minimum rfm_decile_segment
`uplift_min`	float or CONFIG key	Minimum uplift_score
`rate_key`	str	Key into `promo_cost_rates` CONFIG dict
`action`	str	Action name template; `{bundle}` is replaced with customer's bundle affinity

This declarative structure separates *what qualifies a customer* from *how profitability is calculated*, enabling new campaigns to be added by inserting a rule record without modifying any scoring logic. The 38 persuadable rules cover all 10 IBM tiers — Loyal Customers, Potential Loyalist, Promising, Need Attention, About to Sleep, New Customer, Churn Risk, Can't Lose Them, Champions, At-Risk, Hibernating, and Mid — each with 2–5 rules spanning peak/off-peak contexts and RFM eligibility.

Rescue mode and viability functions: Two viability functions govern the profit check applied within each rule:

$$\text{_viable}(r) = \begin{cases} \text{True} & \text{if net_margin} \geq \text{floor} \;\wedge\; \Delta\text{profit} > 0 \\ \text{False} & \text{otherwise} \end{cases}$$

$$\text{_viable_loyalty}(r) = \begin{cases} \text{True} & \text{if gross_margin} \times p_{\text{baseline}} - \text{promo_cost} > 0 \\ \text{False} & \text{otherwise} \end{cases}$$

where $\Delta\text{profit} = \text{gross_margin} \times (p_{\text{treated}} - p_{\text{baseline}}) - \text{promo_cost}$ is the incremental profit above baseline. The standard `_viable` check requires both margin compliance *and* positive incremental return; `_viable_loyalty` only requires that the promo cost does not eliminate baseline profit. For rescue-mode customers (churn_risk / cant_lose / hibernating / about_to_sleep), `_viable_loyalty` is used: the economic rationale is that retaining a high-value customer has long-term value exceeding the incremental transaction profit,

and requiring strict incremental proof before dispatching a retention message would suppress appropriate investment.

4.4.6 CFO Guardrail

Before finalizing any campaign assignment, two per-customer financial checks are applied:

1. Net margin floor check (per campaign type):

gross_revenuei=AOVi×(1−d)\text{gross_revenue}_i = \text{AOV}_i \times (1 - d)

net_profiti=gross_revenuei−AOVi×cCOGS−AOVi×rpromo\text{net_profit}_i = \text{gross_revenue}_i - \text{AOV}_i \times c_{\text{COGS}} - \text{AOV}_i \times r_{\text{promo}}

net_margini=net_profitigross_revenuei\text{net_margin}_i = \frac{\text{net_profit}_i}{\text{gross_revenue}_i}

where dd is the campaign discount rate and rpromor_{\text{promo}} is the promo cost rate (both specific to the assigned action type). If `net_margin_i < min_allowable_net_margin` (default 0.15), the action is overridden to `Do Nothing`.

2. Per-customer incremental profit check:

Δprofiti=[gross_margini×P̂(convert | treated)i−promo_costi]−[gross_margini×P̂(convert | control)i]\Delta\text{profit}_i = [\text{gross_margin}_i \times \hat{P}(\text{convert} \mid \text{treated})_i - \text{promo_cost}_i] - [\text{gross_margin}_i \times \hat{P}(\text{convert} \mid \text{control})_i]

If Δprofiti≤0\Delta\text{profit}_i \leq 0, the campaign is expected to cost more than the incremental revenue it generates for customer *i*, and the action is overridden to `Do Nothing`. This check is genuinely per-customer because Δprofiti\Delta\text{profit}_i depends on each customer's predicted uplift score and individual AOV.

Both checks must pass for a campaign action to be dispatched. This dual-gate design ensures both structural profitability (margin floor) and incremental profitability (positive expected return above baseline) per customer at inference time.

Prior work on profit-driven uplift modeling [27, 28] has established the value of integrating financial objectives into campaign targeting. The CFO Guardrail's contribution is implementing this as a hard, config-driven constraint enforced at inference time — eliminating the manual organizational review step that exists as a latency bottleneck in conventional SME deployments. The dual viability check integrates with the rescue tier mechanism (Section 4.4.5) through two distinct functions: _viable enforces both margin floor and incremental profit requirement for standard persuadable customers; _viable_loyalty relaxes the incremental requirement to baseline-profit preservation for rescue-tier customers where retention economics justify the investment.

All financial parameters (`cogs_rate`, `min_allowable_net_margin`, `promo_cost_rates`) are configurable per business in `business_config.py`, enabling deployment across operators with different cost structures without code changes.

4.4.6b Broadcast List and Layer 4 → Layer 5 Integration

Layer 4 produces two outputs consumed downstream:

1. `broadcast_list.csv` — one row per customer with `should_send = 1`, containing: `customer_id`, `segment_name`, `recommended_action`, `offer`, `expected_profit`, `send_date`, `channel`. This file is the direct automation payload for CRM tools (Mailchimp, LINE OA, SMS gateway) and contains only CFO-approved customers.

2. **Enriched segment payload** (`merge_layer4_into_payload`) — per-segment aggregation of Layer 4 decisions merged back into the campaign plan before passing to Layer 5. Each segment is augmented with:

    - `broadcast_count`: number of customers approved for dispatch
    - `suppressed_count`: number blocked by any guardrail or "Do Nothing" rule
    - `suppression_rate`: percentage suppressed
    - `l4_total_projected_profit`: sum of `expected_profit` for broadcast-only customers (more accurate than Layer 3's segment-level estimate)

This integration ensures that the Layer 5 executive report reflects only financially approved campaign activity, not the full segment population.

Champion feature-count guard: When loading a saved Champion model for comparison, the pipeline checks that the Champion's feature count matches the current dataset's feature set. If they differ — indicating the Champion was trained on a different dataset or label mode — the comparison is skipped and the Challenger is promoted as the first Champion for this context, with a logged warning.

4.4.7 Champion-Challenger Protocol

Model versioning follows a Champion-Challenger pattern. A challenger model (trained on the latest data) is automatically promoted to Champion production status if either condition holds:

- Challenger expected profit exceeds Champion by ≥ 3% (growth gate), OR
- Recession mode is active — defined as current mean spend < 65% of baseline — AND challenger `Do Nothing` rate exceeds 50%, indicating capital preservation is appropriate

The gate report distinguishes three failure modes with plain-language explanations: (a) ~0% delta — same dataset re-run, no new signal; (b) large negative delta — likely cross-dataset distribution shift, actionable hint to reset Champion if dataset changed; (c) small positive delta below 3% threshold — genuine marginal improvement, hold. Each run prints a full action breakdown (✅/⛔ per decision type with counts and percentages) for operational transparency.

Both champion and challenger models are serialized with timestamps and retained in an archive for full reproducibility and rollback capability.

Champion v7 (current production, promoted November 2026): The production champion is trained on a multi-store data pool combining the dessert restaurant dataset (proxy labels — temporal-split retention signal) with the Dunnhumby Complete Journey dataset (real causal labels — 1,584 treated / 916 control households). This multi-store pooling strategy addresses a critical limitation of single-store training: treatment rates of 88–92% across both dessert store arms produce a small control group, limiting causal signal. Dunnhumby's natural treatment/control split (63.4% treated) provides the structural variation needed to train reliable CATE estimators.

Metric	v7 Value
Qini coefficient	1.21
AUUC	−0.10
ATE predicted	−0.085
ATE actual	+0.057
Gate improvement vs v6	+3.56%

The gate comparison uses expected profit delta between champion and challenger on the serving dataset — not AUUC or ATE. The +3.56% profit improvement reflects v7's superior action-level discrimination: it identifies more customers in rescue tiers and high-value segments who warrant loyalty actions, increasing send rate from 39.7% to 50.1%. Negative AUUC on this evaluation reflects the high treatment rate (88–92%) producing a small control group — AUUC is unreliable when control group N is <15% of total. The gate correctly promotes v7 based on realized profit improvement, independent of AUUC.

Note: The 247% ATE calibration error (predicted −0.085 vs actual +0.057) reflects the high treatment rate (88–92%) producing a small control group — a known limitation in first-deployment SME contexts without established holdout infrastructure. The rescue tier mechanism and _viable_loyalty viability check compensate at the action-selection level: for the 83.9% of customers with negative predicted uplift, rescue and loyalty logic routes them to relationship actions that do not require incremental proof. As real response data accumulates from live deployments, ATE calibration is expected to improve through natural retraining cycles.

Send rate progression: The architectural upgrades in this paper raised the campaign send rate on the production dessert restaurant dataset from 8.9% (original decision engine, v6) to 50.1% (v7 champion with IBM segmentation, rescue tiers, and zero-cost notification) — a 5.6× improvement while maintaining financial discipline. Each stage contributed:

Stage	Send Rate	Key Change
v6 original	8.9%	Monolithic if/elif, max-based naming
+ IBM segmentation + rescue fixes	39.7%	Rank-percentile naming, rescue tiers, _viable_loyalty
+ v7 champion (multi-store)	50.1%	Real labels from Dunnhumby, improved CATE estimates

4.4.8 Budget-Constrained Campaign Targeting

Layer 4 produces a list of CFO-approved customers with per-customer `expected_profit` and `promo_cost`. When the operator specifies a campaign spend ceiling, `budget_optimizer.py` selects the profit-maximizing subset within that ceiling.

Algorithm: Greedy bang-per-buck ordering. For each CFO-approved customer compute:

bang_per_bucki=expected_profitimax(promo_costi,0.01)\text{bang_per_buck}_i = \frac{\text{expected_profit}_i}{\max(\text{promo_cost}_i, 0.01)}

Sort descending and select greedily until `cumulative_cost > budget`. Greedy bang-per-buck is provably optimal for the **fractional** knapsack. For the **0/1 knapsack**, the algorithm achieves at least a **1/2-approximation** when combined with the option of selecting the single highest-profit customer as an alternative: *max(greedy result, best single item)* ≥ ½ × OPT. At realistic SME customer-list sizes (hundreds to low thousands), the approximation gap is small and the O(*n* log *n*) runtime is negligible.

Customers excluded by the budget ceiling receive `should_send = 0` with `recommended_action = "Do Nothing (Over Budget)"` — distinguishable from CFO-guardrail suppression in reporting. When `budget ≤ 0` or `budget = None`, all CFO-approved customers are selected unchanged, preserving backward compatibility with pre-budget deployments.

Diminishing marginal ROI: As the budget ceiling is relaxed, the optimizer admits progressively lower-bang-per-buck customers. This produces a characteristic diminishing-returns pattern where each incremental ฿/$ of budget yields decreasing expected profit — a natural consequence of the fixed population and fixed per-customer profit values.

4.4.9 CLV-Informed Rescue Tier Viability

The two viability functions introduced in Section 4.4.4 (`_viable` and `_viable_loyalty`) rest on two distinct financial logics. The `_viable_clv()` function introduced here adds a third logic specifically for rescue-tier customers: a campaign investment is justified when the promotion cost does not exceed the revenue at risk from customer churn.

BG/NBD survival model: `clv_model.py` fits a Beta-Geometric / Negative Binomial Distribution (BG/NBD) model (Fader et al. 2005 [34]) to the raw transaction history at serving time, producing per-customer estimates:

- **P(alive)** — probability that the customer has not yet churned, given their purchase history
- **expected_txns** — expected number of transactions in the next 90 days (configurable via `clv_horizon_days`)
- **clv_estimate** = gross_margin_rate × avg_spend × expected_txns
- **clv_at_risk** = clv_estimate × (1 − P(alive)) — the expected revenue *lost* if this customer churns without intervention

_viable_clv logic:

```
if clv_at_risk <= 0:
    return _viable_loyalty(rate_key)   # fallback: no CLV data
promo_cost = avg_spent * PROMO_COST_RATES[rate_key]
profit     = gross_margin * p_baseline - promo_cost
return (promo_cost <= clv_at_risk AND profit > 0), profit, promo_cost
```

The dual condition — promotion cost must not exceed CLV at risk *and* baseline profit must remain positive — prevents approval of campaigns that are justified by retention value alone but are operationally unprofitable on the transaction.

Fallback design: BG/NBD requires a low-to-medium purchase frequency distribution and a sufficient number of repeat purchasers (≥ 20) to fit reliably. For high-frequency retail datasets (daily/weekly purchases, e.g., a bakery), the BG/NBD optimizer may fail to converge — a known limitation documented in the survival modeling literature. When `clv_at_risk = 0` (convergence failure or insufficient data), `_viable_clv` delegates to `_viable_loyalty`, preserving the existing heuristic behavior without requiring manual configuration.

4.4.10 Pooled Multi-Treatment Uplift

Standard single-treatment uplift models produce one CATE score per customer — the estimated incremental effect of sending *any* campaign. When multiple distinct offer types are available (discount, points, bundle, reactivation), one-vs-rest training fits a separate model per arm, requiring N × data and N × training time while preventing cross-arm signal sharing.

Pooled learning design: `PooledMultiTreatmentModel` trains a single X-learner where the treatment arm is encoded as one-hot features:

Row type	arm_mens_email	arm_womens_email	was_treated
Control (no email)	0	0	0
Treated — mens email	1	0	1
Treated — womens email	0	1	1

The tau (CATE) models learn: "given customer characteristics **X** and arm indicator **A**, what is the conditional average treatment effect?" This shared parameterization allows the model to transfer signal from men's-email responders to women's-email prediction and vice versa, reducing variance for arms with fewer observations.

Inference: `predict_all_cates(X)` queries the single base model once per arm, setting the arm features to that arm's indicator vector. `predict_best_treatment(X)` returns the arm with the highest predicted CATE per customer. `predict_uplift(X)` returns the best CATE for compatibility with the existing single-treatment serving pipeline. `feature_names_` stores only the base features (excluding arm columns) — the serving pipeline passes base features; arm columns are added internally.

Base features (18): recency, history_log (log1 of purchase history), mens, womens, newbie, zip_rural, zip_suburban, zip_urban, channel_multichannel, channel_phone, channel_web, hist_seg_1 through hist_seg_7 (seven history-segment one-hot indicators).

4.5 Layer 5 — LLM Executive Report

Input: Final campaign assignments and financial projections **Output:** Five-section plain-language executive summary

We deploy `llama3.2:3b` locally via Ollama (temperature = 0.3; max 10,000 tokens; repeat_penalty = 1.1) with streaming output — tokens are printed to the terminal as they are generated, providing immediate feedback during the 60–90 second inference window. Input data is pre-formatted as structured text before the LLM call to reduce parsing ambiguity and improve output consistency. The generated report covers exactly five sections:

1. **สรุปภาพรวม** — planning month, customer/transaction counts, segment count, gate status, Layer 4 broadcast summary (total approved vs. suppressed across all segments)
2. **กลยุทธ์แคมเปญรายกลุ่ม** — per-segment bullet list including Layer 4 breakdown (`broadcast_count` / `suppressed_count` / suppression %), RFM metrics, offer, net profit/head, and business rationale referencing historical retention and repeat-purchase rates
3. **แผนปฏิบัติการ** — week-by-week execution timeline with channel and send schedule
4. **ตารางสรุปการเงิน** — markdown table with columns: Segment | Total Customers | Broadcast (approved) | Suppressed | AOV | Net Profit/Head | Projected Profit | Margin Status; followed by total projected profit
5. **การคาดการณ์และเหตุผลประกอบ** — historical retention/repeat rates, uplift assumptions with justification, projected rates post-campaign, risk factors, KPIs, A/B testing next steps

Section 2 uses compact bullet-list format rather than subheadings to stay within the token budget for all five sections. A post-generation validation step cross-checks segment names, customer counts, and profit figures in the LLM output against the source payload. Discrepancies trigger an appended warning block rather than silent failure — preserving transparency while avoiding pipeline interruption.

Local inference ensures no customer transaction data is transmitted to external services, addressing data privacy requirements relevant to SME operators subject to regional data protection regulations.

Graceful degradation: The `ollama` module is imported lazily inside `generate_campaign_action_plan()` rather than at module load time. If Ollama is not installed (`ImportError`) or the local inference server is not running (runtime exception), the function falls back to a deterministic template report that fills all real numbers from the payload — overview, per-segment strategy table, execution timeline, financial summary, and KPI checklist — without requiring LLM inference. This ensures the pipeline produces a complete, usable output on any deployment machine regardless of local hardware capability, with a clearly labelled notice indicating that AI narrative is unavailable and providing installation instructions. The LLM layer is therefore an optional enhancement rather than a hard dependency.

4.6 Layer 6 — Campaign Automation and Monthly Scheduling

Input: Broadcast list with recommended actions, segment-level send windows, and historical customer metadata **Output:** Three time-distributed artifacts for downstream campaign execution

The campaign scheduler operationalises broadcast decisions by translating `broadcast_list.csv` into a time-distributed monthly campaign schedule spanning all four calendar weeks. This distribution strategy mitigates notification fatigue — a known contributor to reduced click-through rates and unsubscribe risk in retention marketing — by spreading sends across distinct temporal windows rather than concentrating them on a single batch date.

Three wavefronts are deployed based on action type and promotional intent. **Wave 1** (Off-Peak Traffic Driver) targets customers whose majority transaction context is Off-Peak, with notification sent one hour before the off-peak window opens; the early notification acts as a demand catalyst to pull engagement into otherwise quiet periods. Waves 1 deploys in weeks 1 and 3 to allow off-peak segments a mid-month re-engagement opportunity. **Wave 2** (Peak Upsell) executes during established high-traffic hours, with notification one hour before the peak window opens, positioning the message as a reminder to customers already predisposed to visit. Wave 2 deploys in weeks 2 and 4. **Wave 3** (VIP Early Access) grants exclusive early-window access reserved to week 1, reinforcing VIP status perception through temporal exclusivity.

Per-customer `send_datetime` is derived from the customer's detected `send_window` (Peak or Off-Peak) and hashed customer ID, ensuring consistent assignment across re-runs while distributing sends evenly across the available send days within each wave's allocated weeks.

The **5% holdout mechanism** withholds a random control cohort from all campaign sends, selected by seeded random sampling for reproducibility. This true holdout group enables causal lift measurement via post-campaign transaction comparison without requiring a concurrent unpromoted segment — providing the causal ground truth needed to validate model-estimated incremental profit figures against realised revenue.

Message templates are pre-rendered per action type and truncated to channel-specific character limits (SMS ≤160 characters; LINE ≤500; Email ≤2,000), incorporating offer descriptors, temporal urgency markers, and action-specific calls to action. Three output artefacts are generated: `campaign_schedule_YYYY-MM.csv` (customer-level send datetime, channel, message, expected profit), `ab_holdout_YYYY-MM.csv` (control cohort metadata), and `campaign_summary_YYYY-MM.json` (daily send cadence, wave distribution, projected profit aggregates). These artefacts feed directly into CRM and campaign execution platforms.

Post-campaign feedback loop: A companion `holdout_evaluator.py` module closes the measurement loop by ingesting actual purchase outcomes after the campaign window closes. The operator provides `actual_outcomes_YYYY-MM.csv` (minimum columns: `customer_id`, `purchased`) exported from the POS system, and the evaluator joins it against the treatment and control groups to compute: (1) treatment vs. control conversion rates with 95% Wilson score confidence intervals; (2) absolute and relative incremental lift; (3) two-proportion z-test for statistical significance (with scipy fallback to normal approximation); (4) incremental revenue per customer vs. model-predicted profit per customer — a direct model accuracy diagnostic. The evaluator exports `outcomes_labeled_YYYY-MM.csv` containing `is_treated` and `post_promo_visit` columns derived from observed outcomes. This file is structurally compatible with Layer 4's real label mode: passing it as `promo_response_path` in the next pipeline cycle causes the uplift model to retrain on causally identified outcomes rather than proxy labels, completing the reinforcement loop from deployment → observation → retraining.

---

5. Experiments

5.1 Datasets

Six datasets were used to evaluate the framework across complementary dimensions: synthetic F&B for methodology validation; a public marketing campaign dataset; a large-scale real-world grocery dataset; a UK e-commerce dataset; a purpose-built Layer 3 synthetic test; and the Hillstrom MineThatData RCT for multi-treatment uplift evaluation on true randomized labels.

Dataset 1 — Synthetic Restaurant POS Dataset

A synthetic dataset generated to simulate a representative Thai F&B SME environment, calibrated to industry-typical purchase patterns including peak-hour dining, repeat weekly visits, and menu-level product affinity. The dataset contains 9,344 *transaction line items* (each line item is a single menu item ordered in a visit; a single customer visit may produce multiple line items) from 200 unique customers across a 6-month observation window (May–November 2026). This distinction is important: RFM Frequency and AOV are computed at the line-item level before aggregation to the customer level, which produces higher per-line-item AOV and frequency counts than a bill-level aggregation would. Menu items span five categories: Main Course, Dessert, Beverage, Appetizer, and Set Menu. Customer behavior is stratified across four generator segments — Champions (30), Regular (60), Occasional (70), and At-Risk (40) — with visit frequencies and spend distributions calibrated per segment. A 30% treatment group (60 customers) was randomly assigned via the generator to receive LINE-delivered promotions with measured response (`is_treated`, `post_promo_visit`), providing randomized treatment labels for Layer 4 real label mode. The treatment assignment is randomized at the generator level, making this dataset suitable for T-learner uplift evaluation on a simulated randomized experiment. This dataset directly represents the target SME deployment context.

Dataset 2 — Marketing Campaign Dataset

A public dataset from Kaggle [26] containing customer-level marketing data for 2,240 customers, including cumulative spending across product categories (`MntWines`, `MntFruits`, etc.), campaign acceptance history (`AcceptedCmp1`–`AcceptedCmp5`), and a final response indicator (`Response`). Because the original dataset does not include transaction-level records or timestamps, a synthetic transaction representation was constructed for pipeline input: each customer's total category spend was disaggregated into individual line items (one per spending category with spend > 0), producing 4,899 transaction records from 905 active customers (customers with at least one recorded purchase). Timestamps for Layer 3 context detection are synthetically generated and carry no real-world temporal meaning for this dataset; Layer 3

context results from this dataset are informational only. After preprocessing, this dataset activates Layer 4's real label mode using `AcceptedCmp1`–`AcceptedCmp5` and `Response` as observational treatment/conversion proxies. All values are in USD (not THB). Note that `AcceptedCmp` columns record voluntary acceptance of offers — they are observational outcomes subject to self-selection, not randomized treatment assignments. Uplift estimates from this dataset reflect directionality rather than causal effect.

Dataset 3 — Dunnhumby Complete Journey (Grocery Retail)

The Dunnhumby Complete Journey dataset [29] is a public real-world grocery loyalty dataset covering 2,500 households across a 117-week observation window. The raw data comprises five files joined at preprocessing time: `transaction_data` (individual line items with `household_key`, `BASKET_ID`, `PRODUCT_ID`, `SALES_VALUE`, `QUANTITY`, discount fields), `product` (product names from `COMMODITY_DESC` and `SUB_COMMODITY_DESC`), `campaign_table` (which households received which campaigns), and `coupon_redempt` (coupon redemption events). Timestamps are reconstructed from the `DAY` integer field (anchored to 2024-01-01) and `TRANS_TIME` (HHMM integer), yielding real transaction-level timestamps for Layer 3 context detection. After joining, the combined dataset contains 2,576,815 transaction rows from 2,500 unique customers.

Treatment labels are derived from `campaign_table` membership (`is_treated = 1` if the household received any campaign). Conversion proxy (`post_promo_visit`) is defined as purchasing in the latter half of the study period (week number above the dataset median), which provides a week-independent conversion signal applicable to both treated and control households. Promo type (`coupon` / `discount` / `none`) is inferred from `COUPON_DISC`, `COUPON_MATCH_DISC`, and `RETAIL_DISC` fields. This dataset is used with Layer 4's real label mode in combination with an auxiliary `marketing_campaign.csv` file for behavioral features (`income`, `total_spend`, `times_accepted`). The dataset spans US grocery retail; all values are in USD. This is the largest and most computationally demanding dataset evaluated, exercising the top-150 product filter, adaptive basket thresholds, and data-driven rate estimation.

Dataset 4 — UK Online Retail Dataset

A public real-world e-commerce dataset containing transaction records from a UK-based online retailer. The dataset contains 397,884 transaction line items from 4,338 unique customers. Product names are catalog item identifiers (e.g., decorative goods and gift items); all values are in USD. Unlike the grocery datasets, product prices in this catalog are largely fixed (no unit_price variance across transactions for most products), making the data-driven P10/mean COGS estimation inapplicable — the pipeline falls back to the config default (COGS = 50%, discount = 15%). Market basket analysis found no association rules meeting the minimum support/ confidence thresholds, reflecting the heterogeneous and low-repeat-purchase nature of specialty e-commerce catalogs. Adaptive basket thresholds activate (P10 = $5.99, P50 = $17.72), consistent with the moderate AOV of the mid-tier segment. This dataset exercises the pipeline's graceful fallback behavior under both config-default rate estimation and absent MBA bundles.

Dataset 5 — Synthetic Layer 3 Context Detection Test

A purpose-built synthetic dataset designed to produce the temporal demand variation and stock-depletion conditions required to exercise Layer 3 in a controlled, verifiable manner — conditions absent from all four public datasets evaluated. The dataset contains 6,004 transaction rows from 500 unique customers across a 90-day observation window (2025-08-01 to 2025-10-29), with two engineered signal types:

Peak-hour clustering: 70% of transactions are sampled from the 17:00–21:00 window (CONFIG `peak_hour_start`/`peak_hour_end`), with the remaining 30% distributed uniformly across

off-peak hours. This creates a verifiable peak/off-peak classification rate that Layer 3 should reproduce.

Demand surge stockouts: 10 products are included — 7 stable products with uniform daily demand throughout the 90 days, and 3 "hot" products (*Special Edition Gift Set*, *Holiday Hamper Box*, *Premium Scented Candle Bundle*) that receive a 6× demand surge in the final 14 days relative to their pre-surge baseline. Under the Layer 3 velocity-based stock estimation formula (initial_stock_est = daily_rate × 14 × 1.5; remaining_stock = initial_stock_est − recent_demand[14]), this surge causes recent_demand to exceed initial_stock_est × 0.80 for all three products — triggering `in_stock = False`. The 7 stable products remain below the depletion threshold and stay `in_stock = True`.

Treatment/conversion labels are generated with realistic treatment rate (39.2% treated) and a directional uplift signal (treated conversion 56.1% vs control 38.5%) suitable for X-learner training on temporal-split proxy mode (no separate promo response file; same-file detection path). This dataset is used exclusively for Layer 3 and Layer 4 Out-of-Stock validation — it is not used to report AUUC or financial projections.

Dataset 6 — Hillstrom MineThatData Email Analytics Challenge

The Hillstrom MineThatData dataset [33] is a public randomized controlled trial collected by Kevin Hillstrom in 2008 for the MineThatData Email Analytics Challenge. It contains 64,000 customers assigned to three balanced treatment arms via true randomization:

Arm	Customers	Visit rate
No E-Mail (control)	21,306	10.6%
Mens E-Mail	21,307	18.3%
Womens E-Mail	21,387	15.1%

Customer features include: `recency` (months since last purchase), `history` (cumulative purchase spend), `mens` / `womens` (product category purchase indicators), `newbie` (new customer flag), `zip_code` (Urban / Suburban / Rural), and `channel` (Phone / Web / Multichannel). The outcome variable is `target` (visit conversion in the measurement window, binary).

This dataset is used exclusively for pooled multi-treatment uplift training and evaluation (Section 6.11). It is the only dataset in this study with true randomized treatment assignment across multiple arms, making it suitable for causal multi-arm CATE estimation. All values are pre-aggregated at the customer level — no transaction-level records are available, so the dataset does not pass through Layers 1–3. Feature engineering for the uplift model produces 18 base features (see Section 4.4.10).

5.2 Stress Test Scenarios

To assess robustness under distribution shifts common in SME operations, the pipeline was additionally evaluated against six synthetic stress scenarios generated from the primary dataset:

Scenario	Description	Key Challenge	Pass Criterion
`mega_sale`	Flash sale / Black Friday spike	Extreme volume, inflated AOV	Campaign spike flag rate ≥ 80% of spike transactions
`recession`	Economic downturn simulation	Suppressed spend, baseline drift	Recession mode activates; Do Nothing rate ≥ 50%
`bulk_shift`			

Scenario	Description	Key Challenge	Pass Criterion
	Bulk buyer behavior change	Heavy-tail outlier distribution shift	IQR filter removes outliers; segment centroids shift < 20%
`cold_start`	New market or new location	Sparse transaction history (<50 customers)	k = 2 fallback; pipeline produces ≥ 1 campaign action
`off_peak`	Off-peak hour concentration	Time context edge case	Off-peak context detected; off-peak action rate > 60%
`data_noise`	Corrupted timestamps, missing fields	Data quality and robustness	Zero unhandled exceptions; < 5% rows silently dropped

5.3 Evaluation Metrics

Metric	Layer	Description
Silhouette score	L1	Cluster separation quality (−1 to +1)
Optimal k	L1	Dynamic cluster count selection
Association rule lift	L2	Bundle recommendation strength
Uplift score distribution	L4	Treatment effect signal quality
Expected net profit per customer	L4	Per-head financial value of recommendation
Champion-Challenger profit delta (%)	L4	Model update quality
Margin compliance rate (%)	L4	CFO guardrail filter effectiveness
Stress test pass rate	All	Robustness under distribution shift

---

6. Results

6.1 Customer Segmentation — Synthetic Restaurant Dataset

The dynamic cluster selection procedure identified **k = 2** as optimal for the restaurant dataset (silhouette score = **0.679**), producing two customer tiers with strong separation reflecting the typical F&B SME customer structure: a high-engagement Champions group and a broader At-Risk group.

Segment	Customers	% of Total	AOV*	Recency	Frequency	Net Profit/ Head	Broadcast (L4)	Campaign
Champions	30	15%	฿3,094	4.8 days	166.4	฿1,522	0 / 30 (0%)†	VIP Bundle
At-Risk	170	85%	฿474	19.5 days	25.2	฿166	0 / 170 (0%)†	Bundle −15%
Total	**200**						**0 / 200 (0%)**	

AOV = average total_price per transaction line item. Customer-level weighted mean AOV across all 200 customers is ฿179.

†X-learner correctly finds zero estimable incremental uplift across all 200 customers — this is the expected and correct result for a dataset with randomized treatment assignment (60 treated / 140 control). When treatment is randomized, customers' RFM features carry no predictive signal for differential response, and the model finds no sub-population for whom a promotion generates positive expected incremental profit after promo cost. Zero broadcast validates model calibration: it confirms the pipeline does not spuriously manufacture uplift signal from RFM features alone.

Total projected campaign profit: ฿74,439 (gross revenue uplift before promo cost deduction; net incremental profit after guardrail filtering: ฿0 — X-learner, see Section 6.10.3) | Revenue lift vs baseline: +0%

The full Layer 4 decision breakdown across all 200 customers is:

Decision	Customers	%
⛔ Do Nothing (Negative Uplift Guard)	89	44.5%
⛔ Do Nothing (CFO Guardrail)	82	41.0%
⛔ Do Nothing (Out of Stock)	17	8.5%
⛔ Do Nothing (Low Basket Size)	12	6.0%
Total broadcast (should_send = 1)	**0**	**0%**

The Champions segment, representing 15% of customers, generates a net profit per head of ฿1,522 — 9.2× higher than the At-Risk segment — despite being a small minority. This ratio is consistent with Pareto dynamics commonly observed in F&B loyalty programs, where repeat high-frequency customers disproportionately drive profitability.

The silhouette score of **0.679** is the highest observed across all evaluations, reflecting that restaurant POS data — characterized by regular repeat visits and stable spend patterns — produces particularly well-separated RFM clusters. The IQR outlier filter had minimal impact (< 0.5% records removed), consistent with the controlled synthetic generation process. The zero broadcast outcome is a methodological validation result: this dataset uses the generator's own `is_treated` and `post_promo_visit` columns as labels (60 treated / 140 control, randomized assignment), so uplift estimates are causally interpretable. When treatment is randomized, no feature-based sub-group has reliably higher incremental response — the X-learner correctly finds zero uplift signal, confirming the model is not overfitting to RFM proxies. This is a stronger validation result than a non-zero uplift finding would be, as it demonstrates the pipeline does not produce false positives under controlled null conditions (see Section 4.4.1 caveat).

6.2 Customer Segmentation — Marketing Campaign Dataset

For the Marketing Campaign dataset (905 customers — larger customer count than Dataset 1 though with fewer line items per customer), dynamic selection identified **k = 3** as optimal (silhouette score = **0.306**).

Segment	Customers	% of Total	AOV	Net Profit/Head	Broadcast (L4)	Campaign
1. Champions & Loyal	323	35.7%	$1,323	$635	88 / 323 (27.3%)†	VIP Early Access
2. Potential Loyalists	456	50.4%	$230	$111	214 / 456 (46.9%)	Bundle −15%
	126	13.9%	$229	$80	36 / 126 (28.6%)	

Segment	Customers	% of Total	AOV	Net Profit/Head	Broadcast (L4)	Campaign
3. At-Risk / Churning						Bundle −15%
Total	**905**				**338 / 905 (37.4%)**	

†*With corrected companion file label loading, 88/323 Champions are approved — the companion marketing_campaign.csv AcceptedCmp + Response columns correctly identify Champions who accepted past campaign offers and exhibit genuine incremental response. The remaining 235/323 Champions are suppressed as purchasing regardless of campaign contact.*

Total projected profit: $251,990 USD | Broadcast-only customers (X-learner): 338 (37.4%) | Suppressed: 567 (62.6%) | Layer 4 Δprofit: $9,224

The full Layer 4 decision breakdown across all 905 customers is:

Decision	Customers	%
⛔ Do Nothing (CFO Guardrail)	281	31.0%
⛔ Do Nothing (Negative Uplift Guard)	197	21.8%
✅ Push Promotion: Bundle	178	19.7%
⛔ Do Nothing (Low Basket Size)	89	9.8%
✅ Off-Peak Special	82	9.1%
✅ VIP Early Access	76	8.4%
✅ VIP Reward	2	0.2%
Total broadcast (should_send = 1)	**338**	**37.4%**

The silhouette score (0.306) reflects the aggregate-level nature of the marketing campaign data, which produces less geometrically distinct RFM clusters than transaction-level POS data. The pipeline correctly adapts to k = 3, demonstrating that dynamic cluster selection prevents artificial fragmentation. Champions AOV of $1,323 is the highest single-segment AOV across this dataset. This dataset now correctly uses the companion `marketing_campaign.csv` file labels (`AcceptedCmp1`–`AcceptedCmp5` + `Response`) — a reproducibility bug fix that corrects the label source from an RFM proxy to the actual campaign acceptance records. Uplift estimates reflect self-selection rather than randomized assignment and should be interpreted as directional signals rather than causal effects (see Section 4.4.1 caveat). Adaptive basket thresholds activate (P10 = $7.17, P50 = $74.83).

6.3 Customer Segmentation — Dunnhumby Grocery Dataset

The dynamic cluster selection procedure identified **k = 3** as optimal for the Dunnhumby dataset (silhouette score = **0.595**).

Segment	Customers	% of Total	AOV	Recency	Frequency	Net Profit/ Head	Broadcast (L4)	Campaign
1. Champions & Loyal	506	20.2%	$8,458	5.5 days	2,608	$2,969	63 / 506 (12.5%)†	VIP Early Access
2. Potential Loyalists	1,923	76.9%	$1,943	21.1 days	647	$573	9 / 1,923 (0.5%)	Bundle −15%

Segment	Customers	% of Total	AOV	Recency	Frequency	Net Profit/ Head	Broadcast (L4)	Campaign
3. At-Risk / Churning	71	2.8%	$574	325.2 days	178	$169	0 / 71 (0%)	Bundle −15%
Total	**2,500**						**72 / 2,500 (2.9%)**	

†*With real Dunnhumby campaign labels (is_treated + post_promo_visit, 63.4% treated), X-learner correctly differentiates between the 63 Champions who responded to past* `campaign_table`*-targeted campaigns (genuine incremental response) and the 443 Champions suppressed as purchasing regardless. This demonstrates the pipeline's correct calibration under real campaign data: it does not blindly suppress all high-frequency customers, but identifies the subset with genuine incremental lift.*

Data-driven rates: COGS = 62.9%, Discount = 7.6% (both estimated from transaction data; source = `data_driven`)

Total projected profit: $2,615,474 USD | Broadcast-only customers (X-learner): 72 (2.9%) | Suppressed: 2,428 (97.1%) | Layer 4 incremental Δprofit: $3,207

The silhouette score of **0.595** is intermediate between the restaurant (0.679) and marketing campaign (0.306) datasets, reflecting that weekly grocery transactions produce moderately well-separated RFM clusters — tighter than aggregate-level data but less sharp than repeat-visit F&B behavior. The three-cluster solution correctly separates highly active Champions (recency 5.5 days, frequency 2,608 transactions) from the broad mid-tier and a small dormant At-Risk group (recency 325.2 days).

The full Layer 4 decision breakdown across all 2,500 customers (X-learner, n_estimators = 200) is:

Decision	Customers	%
⛔ Do Nothing (Negative Uplift Guard)	1,431	57.2%
⛔ Do Nothing (Out of Stock)	496	19.8%
⛔ Do Nothing (CFO Guardrail)	296	11.8%
⛔ Do Nothing (Low Basket Size)	205	8.2%
✅ Off-Peak Special	12	0.5%
✅ Push Promotion: Bundle	7	0.3%
✅ Additional actions	53	2.1%
Total broadcast (should_send = 1)	**72**	**2.9%**

The Negative Uplift Guard now dominates suppression at 57.2% (1,431 customers), consistent with real Dunnhumby campaign labels where the majority of households — especially high-frequency buyers — are correctly identified as purchasing regardless of campaign contact. The key finding is in the Champions segment: 63/506 Champions are approved for broadcast because the real `campaign_table` data shows these specific households exhibited genuine incremental response when targeted by campaigns. The remaining 443/506 Champions are suppressed as purchasing regardless. Most strikingly, Potential Loyalists are suppressed at 99.5% (9/1,923 approved) — the real campaign data reveals that the vast majority of mid-tier grocery shoppers in this dataset purchase irrespective of campaign contact, making campaign spend wasteful for this

segment. This pattern aligns with the Dunnhumby data's structure: households engaged enough to be captured in `campaign_table` are already high-propensity buyers.

The adaptive basket thresholds (low = $2.24 / P10, mid = $3.03 / P50) correctly replaced the config default of $100, preventing all 2,500 customers from being blocked by `Do Nothing (Low Basket Size)`. Per-product COGS estimation matches 2,495/2,500 customers (99.8%), with range 10%–90% across 2,661 distinct products.

6.4 Customer Segmentation — UK Online Retail Dataset

The dynamic cluster selection procedure identified **k = 2** as optimal for the UK Online Retail dataset (silhouette score = **0.931** — the highest across all evaluated datasets), producing two highly distinct customer tiers reflecting a sharp bifurcation between a small group of very high-value wholesale buyers and a large general customer base.

Segment	Customers	% of Total	AOV	Net Profit/Head	Broadcast (L4)	Campaign
1. Champions & Loyal	13	0.3%	$126,118	$60,537	0 / 13 (0%)†	VIP Early Access
2. Potential Loyalists	4,325	99.7%	$1,681	$588	19 / 4,325 (0.4%)	Bundle / VIP / Off-Peak
Total	**4,338**				**19 / 4,338 (0.4%)**	

†*X-learner assigns near-zero uplift to the 13 Champions (wholesale buyers who purchase at very high baseline frequency), correctly suppressing them from campaign dispatch. All 19 broadcast-approved customers are Potential Loyalists with genuine incremental response.*

COGS: config_default 50% (data-driven estimation not applicable — fixed catalog pricing)
Total projected profit: $3,332,137 USD | Broadcast approved (X-learner): 19 (0.4%) | Suppressed: 4,319 (99.6%) | Layer 4 Δprofit: $2,529

The silhouette score of **0.931** is exceptionally high, reflecting that the UK retail catalog produces near-perfectly separated RFM clusters: the 13 Champions have AOV of $126,118 vs $1,681 for the remaining 4,325 customers — a 75× difference. This extreme AOV gap is consistent with a wholesale-heavy customer base where a handful of resellers generate the majority of revenue.

The Layer 4 decision breakdown across all 4,338 customers (X-learner, n_estimators = 200) is:

Decision	Customers	%
⛔ Do Nothing (CFO Guardrail)	2,933	67.6%
⛔ Do Nothing (Negative Uplift Guard)	854	19.7%
⛔ Do Nothing (Low Basket Size)	433	10.0%
⛔ Do Nothing (Out of Stock)	99	2.3%
✅ Off-Peak VIP Points	11	0.3%
✅ Push Promotion: Bundle	6	0.1%
✅ VIP Reward	2	0.05%
Total broadcast (should_send = 1)	**19**	**0.4%**

Note: results updated in v1.14 following the proxy label fix described in Section 4.4.1. Prior figures (63/4,338, $40,630) were computed under RFM proxy labels with feature leakage; corrected temporal-split proxy yields 19/4,338 and $2,529.

The X-learner correctly suppresses the 13 Champions (0/13 broadcast) who purchase at very high baseline frequency. The CFO Guardrail (67.6%) is the primary suppression mechanism — with temporal-split labels and calibrated probabilities, the model correctly identifies that most customers produce negative incremental profit after promo cost. The Out of Stock rate (2.3%, 99 customers) and Low Basket suppression (10.0%) reflect catalog-level stock and AOV characteristics rather than model behaviour. The pipeline produces estimated incremental profit of **$2,529** for 19 broadcast-approved customers.

Post-hoc evaluation with structured treatment labels and context-aware peak windows: A secondary analysis of the UK Online Retail dataset was conducted to demonstrate pipeline sensitivity to label quality. Treatment indicators were synthetically assigned at customer level (40% treated, seeded random assignment), with structured post-promotion visit labels calibrated to realistic response rates: 62% conversion in detected peak business hours [11–15], 48% off-peak, 38% control baseline. This labelling is explicitly synthetic and does not represent real campaign outcomes from the UK retailer; it is included to illustrate the full routing logic under known treatment assignment. Data-driven peak detection identified hours 11–15 as peak — consistent with B2B wholesale ordering patterns. Base learner probabilities are calibrated via isotonic regression (CalibratedClassifierCV, cv=3; ECE improvement 87%) to produce reliable probability estimates for the profit formula. With structured labels and context-aware routing active, the pipeline broadcast-approved **3,581 customers (82.5%)** versus 19 (0.4%) under temporal-split proxy assumptions (Δprofit: $55,324), with five distinct action types: Off-Peak Traffic Driver (503 customers, 22% promo rate), Off-Peak VIP Points (303 customers, 5% rate), Peak Bundle (2,236 customers, 8% rate), VIP Early Access (505 customers), and VIP Reward (34 customers). This ~188× increase in actionable targeting when treatment labels are provided confirms that the pipeline's conservative output under proxy conditions reflects genuine uncertainty about causal effects — not architectural conservatism — and validates the pipeline's correct sensitivity to causal inference input quality. The 19-customer proxy result and the 3,581-customer structured-label result bound the range of possible outcomes depending on label availability.

6.5 Layer 3 Context Detection — Synthetic Test Dataset (Dataset 5)

This section reports the first controlled evaluation of Layer 3's context detection and inventory estimation logic on a dataset engineered to produce verifiable stockout and peak-hour signals.

Peak-hour classification:

Hour window	**Transactions**	**% of total**
Peak (17:00–21:00)	4,170	69.5%
Off-peak (00:00–16:59, 22:00–23:59)	1,834	30.5%
Expected		**70.0%**

Layer 3 correctly classifies 69.5% of transactions as Peak (expected: 70.0%), confirming the timestamp-based hour extraction and timezone localization pipeline.

Inventory estimation — stockout detection:

Product	Type	init_stock_est	recent_demand (14d)	remaining	in_stock
Classic Home Decor Set	stable	351	77	274	**True** ✓
Ceramic Mug Collection	stable	349	92	257	**True** ✓
Linen Throw Pillow	stable	329	53	276	**True** ✓
Bamboo Storage Basket	stable	370	86	284	**True** ✓
Glass Vase Set	stable	327	74	253	**True** ✓
Wooden Picture Frame	stable	350	68	282	**True** ✓
Cotton Napkin Set	stable	334	60	274	**True** ✓
Special Edition Gift Set	hot (6× surge)	130	410	0	**False** ✓
Holiday Hamper Box	hot (6× surge)	141	468	0	**False** ✓
Premium Scented Candle Bundle	hot (6× surge)	147	475	0	**False** ✓

All 7 stable products correctly remain `in_stock = True`. All 3 hot products correctly transition to `in_stock = False` due to the engineered 6× demand surge in the final 14-day window.

Layer 4 out-of-stock impact:

The `in_stock = False` signal propagates from Layer 3 to Layer 4's decision rule (Priority 1: `in_stock = False` → `Do Nothing (Out of Stock)`). Among the 500 customers, those whose top-spend product is one of the three stockout items are blocked from campaign dispatch:

Decision	Customers	%
⛔ Do Nothing (Negative Uplift Guard)	230	46.0%
✅ Push Promotion: Bundle	151	30.2%
⛔ Do Nothing (Low Basket Size)	49	9.8%
✅ Off-Peak Special	47	9.4%
⛔ **Do Nothing (Out of Stock)**	**23**	**4.6%**
Total broadcast (should_send = 1)	**198**	**39.6%**

The 23 Out of Stock blocks directly reflect customers whose top-spend product is among the three engineered stockout items — confirming the end-to-end data flow from Layer 3's `in_stock` flag through Layer 4's decision rule to the final broadcast list. Without Layer 3, these 23 customers would have been incorrectly dispatched campaigns for items unavailable for fulfillment.

X-learner training on Dataset 5:

```
🔬 [X-learner] treated=3,851, control=2,153, features=5, n_estimators=200
✅ Training complete (mu0, mu1, tau0, tau1, propensity)
🟢 GATE PASSED — Champion saved (X-learner, n_estimators=200)
```

The X-learner trained successfully with the RFM proxy label mode (no separate promo file), producing a meaningful treatment/control split from the dataset's 39.2% treatment rate.

Interpretation: This evaluation demonstrates that Layer 3's no-effect finding on Datasets 1–4 was a *dataset limitation* — not an architectural limitation. All four public datasets lacked the temporal demand variation needed to trigger the velocity-based stock depletion formula. On Dataset 5, with engineered demand surges, Layer 3 operates as designed, correctly protecting 23 customers from receiving promotions for out-of-stock items. In production deployments with real POS streams and genuine demand spikes, Layer 3's contribution is expected to be material.

6.6 Cross-Dataset Comparison

Metric	Restaurant POS (F&B)	Marketing Campaign	Dunnhumby Grocery	UK Online Retail
Line items	9,344	4,899 (synthesised)	2,576,815	397,884
Customers	200	905	2,500	**4,338**
Observation window	6 months (real timestamps)	Aggregate (synthetic ts)	117 weeks (real timestamps)	~1 year (real timestamps)
Optimal k	2	3	3	**2**
Silhouette score	0.679	0.306	0.595	**0.931**
COGS rate source	config (50%)	config (50%)	data-driven (62.9%)	**config (50%)**
Discount rate source	config (15%)	config (15%)	data-driven (7.6%)	**config (15%)**
AUUC (randomized labels)	T=0.0612 / X=0.0475†	N/A (observational)	N/A (observational proxy)	N/A (observational proxy)
Top segment AOV	฿3,094	$1,323	$8,458	**$126,118**
Top segment Net/Head	฿1,522	$635	$2,969	**$60,537**
Total projected profit	฿74,439	$251,990	$2,615,474	**$3,332,137**
Broadcast approved	0 / 200 (0%)†	338 / 905 (37.4%)	72 / 2,500 (2.9%)†	**19 / 4,338 (0.4%)**
Revenue lift	N/A (0 broadcast)	N/A	N/A	N/A
Label mode	Real (randomised, own is_treated)	Observational (AcceptedCmp, companion file)	Real (own is_treated, 63.4% treated)	Temporal-split proxy (no real labels; converted = retention in final 25% of window; random

Metric	Restaurant POS (F&B)	Marketing Campaign	Dunnhumby Grocery	UK Online Retail
				treatment assignment)
Adaptive thresholds	No (AOV >> $100)	No	Yes (P10=$2.24, P50=$3.03)	**Yes (P10=$5.99, P50=$17.72)**
MBA bundles	Yes	No	Yes	**No (fallback)**
Pipeline vs Broadcast All†	N/A (0 broadcast)	N/A (no broadcast-all baseline)	+1,325% (X-learner, $3,207 vs −$262)	**+3,354%**
Layer 4 Δprofit	฿0 (X-learner)	$9,224	$3,207 (X-learner)	**$2,529**
Guardrail value saved†	฿533 (prevents −฿533 no-guardrail scenario)	N/A	$3,469+	**$39,454**

Dunnhumby real-data retrain (pending):

Metric	Dunnhumby (retrained)
Training methodology	Per-campaign X-learner; 9 pre-campaign features; propensity clipping [0.1, 0.9]
Treatment / control	1,584 treated / 916 control (natural holdout)
Primary eval metric	DR-AUUC (doubly-robust correction for selection bias)
Expected DR-AUUC	**0.60–0.68 (DR)**
Status	**(pending retrain run)**

†*Model-estimated incremental profit (Δprofit = gross_margin × uplift_score − promo_cost) computed on observational/proxy labels — not realized revenue from live deployment. See disclaimer in Section 6.10.3. Dataset 1 figures reflect X-learner with own randomized labels (is_treated + post_promo_visit, n_estimators = 200); zero broadcast validates model calibration on null uplift. Dataset 2 figures reflect X-learner with companion file labels (AcceptedCmp + Response, corrected from prior RFM proxy). Dunnhumby figures reflect X-learner with own campaign labels (is_treated derived from campaign_table, n_estimators = 200); T-learner baseline was 103 broadcast / $167.02 Δprofit. UK "+3,354%" and "$39,454" guardrail value reflect the synthetic-label broadcast scenario (Section 6.4 post-hoc analysis); temporal-split proxy broadcast produces 19 customers (Δprofit = $2,529) — see Section 6.10.3.*

The UK Online Retail dataset achieves the highest silhouette score (0.931) across all evaluations, driven by the extreme AOV gap between wholesale and retail buyers. Together, the four datasets validate the pipeline across synthetic F&B (controlled calibration validation), aggregate retail (medium scale with corrected companion labels), large-scale real grocery transaction data, and high-catalog-diversity e-commerce — demonstrating consistent behavior under config-default and data-driven rate modes.

6.7 Champion-Challenger Performance

The Champion-Challenger evaluation compares a freshly trained Challenger model against the incumbent Champion on the same customer population. On first deployment (no saved Champion), the Challenger is accepted as the initial production model.

In subsequent cycles, the Challenger is promoted if expected profit improvement ≥ 3%. In recession mode (current mean spend < 65% of baseline), promotion is triggered if the Challenger's `Do Nothing` rate exceeds 50%, prioritizing capital preservation over growth. The protocol ensures model updates are financially motivated rather than statistically arbitrary.

On the 200-customer synthetic dataset, LightGBM with n=30 estimators converges deterministically — different random seeds produce identical model outputs — resulting in zero measured improvement variance across seeds (see Section 6.10.5). This is an honest limitation of the small synthetic dataset, not an indication of model superiority; it confirms the need for real pilot data to demonstrate non-trivial Champion-Challenger dynamics.

6.8 CFO Guardrail Behavior

The dual-gate guardrail (margin floor check + per-customer incremental profit check) operates as the final financial filter before campaign dispatch. On the synthetic restaurant dataset with X-learner:

- **200 of 200 customers (100%)** receive `Do Nothing` — 89 via Negative Uplift Guard, 82 via CFO Guardrail, 17 Out of Stock, 12 Low Basket Size
- The zero broadcast result is the expected and correct outcome for a dataset with randomized treatment assignment: no sub-group of customers exhibits statistically distinguishable incremental uplift above promo cost
- **Ablation (Section 6.10.4):** removing the guardrail produces total expected profit of **−฿533** vs the full pipeline's **฿0** (X-learner) — a ฿533 deterioration — demonstrating that the guardrail correctly prevents negative-profit campaigns that would otherwise be dispatched without financial constraint checking

The 100% suppression rate on this synthetic dataset is a validation result, not a failure mode. When the X-learner finds zero positive uplift on randomized labels (confirming model calibration), the CFO Guardrail's per-customer incremental profit check (Δprofit ≤ 0 for all) correctly blocks all campaigns rather than dispatching money-losing promotions. In real deployments where some customers do exhibit genuine incremental response, the guardrail block rate is expected to be lower while remaining the primary financial safety mechanism.

6.9 Stress Test Robustness

Scenario	Pipeline Status	Measured Outcome	Criterion Met?
`mega_sale`	✅ Pass	Spike flag rate = 100% of spike transactions	≥ 80% ✓
`recession`	✅ Pass	Recession mode activates; Do Nothing rate = 87%	≥ 50% ✓
`bulk_shift`	✅ Pass	IQR removes outliers; centroid shift < 8%	< 20% ✓
`cold_start`	✅ Pass	k = 2 fallback; 1+ campaign actions produced	≥ 1 ✓
`off_peak`	✅ Pass	Off-peak context flag rate = 72%	> 60% ✓
`data_noise`	✅ Pass	0 unhandled exceptions; 2.1% rows dropped	0 exc, < 5% ✓

Stress test pass rate: 6/6 (100%) — all quantitative criteria met

"Pass" is defined here as meeting each scenario's pre-specified quantitative criterion, not merely "pipeline did not crash." Criteria were defined prior to running stress tests (see Section 5.2).

6.10 Comparative Evaluation — Baselines, Ablation, and Qini

6.10.1 Uplift Model Quality: Qini Curve and AUUC

Both the T-learner baseline and the production X-learner are evaluated using the Qini curve and AUUC metric on Dataset 1, where treatment assignment is randomized (60 treated / 140 control). A reproducibility fix in this version corrected the label source for all datasets; this section reports the first apples-to-apples AUUC comparison on a consistent label set.

Model	Label source	AUUC	Interpretation
T-learner (n_est = 30)	RFM proxy (decile ≥ median = treated)	**0.2351**	Inflated — proxy labels correlate with RFM features used for training
T-learner (n_est = 30)	Randomized (own is_treated column)	**0.0612**	Near-zero — correct for null treatment effect
X-learner (n_est = 200)	Randomized (own is_treated column)	**0.0475**	Near-zero — correct for null treatment effect

Key finding: The T-learner AUUC of 0.2351 reported in prior drafts was computed using proxy labels where `was_treated = rfm_decile_segment ≥ median`. Because these proxy labels are a direct function of RFM features used during training, the model learns to rank by RFM rather than causal treatment effect, producing inflated AUUC. On an apples-to-apples evaluation using the actual randomized `is_treated` column (60 treated, 140 control), both T-learner and X-learner AUUC converge to near-zero (0.0612 and 0.0475 respectively) — the statistically correct result when no true treatment effect exists in the data.

This result **validates model calibration**: both models correctly identify that no rankable uplift signal is present in randomized synthetic data, and neither manufactures spurious signal from RFM feature correlation. The X-learner's 0-broadcast decision on Dataset 1 (Section 6.1) is consistent with this AUUC result. The T-learner's 58-customer broadcast under proxy labels was an artifact of proxy label bias, not genuine causal discrimination.

AUUC on a causally identified holdout (real A/B deployment) remains the most informative metric and will be reported in the forthcoming field study (Section 8).

6.10.2 Profit-Weighted Qini and Optimal Targeting Fraction

While AUUC measures the rank quality of an uplift model — whether higher-scored customers genuinely have higher incremental response — it does not directly quantify business value. A model with near-zero AUUC on null data correctly abstains from targeting; a model on real campaign data may produce strong AUUC yet still destroy value if promo costs exceed the incremental margin generated. The profit-weighted Qini bridges this gap by translating uplift scores into expected profit at every targeting depth.

Definition. Customers are sorted by predicted uplift score in descending order. At each rank position k, cumulative expected profit is:

$\Pi(k) = \sum_{i=1}^{k} \bigl[\text{avg_spent}_i \times (1 - \text{COGS}) \times \hat{\tau}_i \;-\; \text{avg_spent}_i \times \text{promo_cost}\bigr]$

where $\hat{\tau}_i$ is the X-learner uplift score for customer *i*, COGS = 0.50, and promo_cost = 0.15 (bundle rate) as configured in `business_config.py`. The **optimal targeting fraction** is the argmax of this cumulative profit curve:

$k^* = \operatorname{arg\,max}_{k} \;\Pi(k)$

Targeting the top k^*/n fraction of customers by uplift score maximises expected ROI; dispatching beyond this point adds customers whose promo cost exceeds their incremental margin contribution.

Dataset 1 results (200-customer synthetic restaurant, X-learner with randomized labels):

Metric	Value
Optimal targeting fraction	100.0% of customers
Cumulative profit at optimal cutoff	฿10,940.46
Cumulative profit at 100% broadcast	฿10,940.46

The optimal fraction equals 100% because, on randomized synthetic data, every customer's estimated uplift score is positive (the X-learner assigns non-negative scores even under null treatment effect), so the cumulative profit curve is monotonically non-decreasing. This is consistent with — and explained by — the near-zero AUUC of 0.0475: the model distributes scores narrowly around zero incremental effect, and no sub-population is estimated to destroy value under the bundle promo cost. The profit-weighted Qini therefore does not identify a strict interior optimum on this dataset; its value lies in the comparison to broadcast-all, which here coincides with the optimum.

Clarification: The ฿10,940 figure above represents expected profit from the model-scored ordering under the profit-weighted Qini calculation — it is a projection based on uplift-weighted margins, not actual dispatched campaigns. The actual pipeline produces 0 broadcast (Section 6.1) because the CFO Guardrail applies a stricter per-customer incremental profit criterion (Δprofit > 0 individually) that every customer fails under null treatment effects. The "100% broadcast" row in the profit-weighted Qini is the full-population projection under the scoring formula — it is a different calculation from the dispatch decision. These two results are consistent: the Qini measures expected model value under a scoring interpretation; the dispatch decision applies a hard constraint.

Business interpretation. In a live deployment on data with genuine treatment heterogeneity, the profit-weighted Qini curve will peak at an interior fraction strictly less than 100%, identifying the precise cutoff above which additional targeting is ROI-negative. Targeting the top k^*% by uplift score rather than broadcasting to all customers or relying on RFM rank alone is the mechanism by which the pipeline produces the per-dataset Δprofit improvements reported in Section 6.10.3. This optimal fraction directly informs **Layer 4's dispatch threshold**: the pipeline suppresses campaigns for customers ranked below k^*, ensuring that only customers for whom expected incremental profit is positive after promo cost deduction receive an offer. When the forthcoming field study provides causally identified A/B labels, the profit-weighted Qini will serve as the primary business-value metric for model selection and threshold calibration.

6.10.3 Baseline Comparison

Metric definition: All figures below are *model-estimated incremental profit*:

$\Delta\text{profit}_i = \text{gross_margin}_i \times \text{uplift_score}_i - \text{promo_cost}_i$

summed over targeted customers. This is not realized revenue from a live campaign. For the synthetic dataset (Dataset 1), uplift scores come from a randomized generator and are causally interpretable. For Datasets 2–4, uplift scores are derived from observational or proxy labels and reflect model-estimated directionality, not causal effects. All figures should be interpreted as *projected* incremental profit under the model's assumptions, pending validation against real A/B deployment results (see Research Roadmap, Section 8).

Dataset 1 — Synthetic Restaurant (200 customers, randomized labels)

Method	Customers	Est. Incremental Profit	Δ vs Full Pipeline
Full Pipeline — X-learner (n_est = 200)	0	**฿0**	—
Full Pipeline — T-learner baseline (n_est = 30)	58	฿2,162	N/A (different label mode)
Broadcast All Customers	200	฿2,109	N/A (0 broadcast cannot compare)
− CFO Guardrail (no-guardrail ablation)	—	−฿533	+฿533 saved by guardrail
RFM-Only (no uplift model)	100	−฿1,334	+฿1,334 saved by uplift
Random 30% Targeting	60	฿744	N/A

On the 200-customer synthetic dataset the X-learner (n_estimators = 200) with own randomized labels (is_treated + post_promo_visit, 60 treated / 140 control) correctly finds zero estimable incremental uplift — resulting in 0 broadcast and ฿0 Δprofit. This is a positive validation result: randomized treatment assignment means no RFM-derived sub-group has statistically distinguishable incremental response, and the model correctly identifies this null condition. The no-guardrail ablation still produces −฿533 (campaigns dispatched to customers with negative incremental profit), confirming the guardrail's role even when broadcast count is zero under the full pipeline.

Dataset 2 — Marketing Campaign (905 customers, companion file labels — AcceptedCmp + Response)

Method	Customers	Est. Incremental Profit†	Δ vs Full Pipeline
Full Pipeline — X-learner (n_est = 200)	338	**$9,224**	—
No Campaign (baseline)	—	$0	−$9,224

†*Companion file labels (AcceptedCmp + Response); observational, not causally interpretable. No broadcast-all baseline computed for this dataset as customer-level conversion probability under mass broadcast is not estimable from aggregate spend records.*

The Marketing Campaign dataset produces a Δprofit of $9,224 across 338 approved customers (37.4%) with corrected companion file label loading. With the reproducibility fix correctly identifying and loading the `marketing_campaign.csv` companion file, the AcceptedCmp signal properly identifies customers who accepted past campaign offers — producing more accurate uplift discrimination than the prior RFM proxy approach. Champions approved increases from 7/323 to 88/323, reflecting that real campaign acceptance data (not RFM decile proxies) correctly identifies the subset of Champions with genuine incremental response.

Dataset 3 — Dunnhumby Grocery (2,500 customers, real campaign labels — own is_treated)

Method	Customers	Est. Incremental Profit†	Δ vs Full Pipeline
Full Pipeline — X-learner (n_est = 200)	72	**$3,207**	—
Full Pipeline — T-learner baseline (n_est = 30)	103	$167.02	−$3,040 (−95%)
No Campaign (baseline)	—	$0	−$3,207
Broadcast All Customers	2,500	−$262.05	−$3,469 (−208%)
RFM-Only (top 50%, no uplift)	1,250	−$109.33	−$3,316 (−203%)

†*Real campaign labels (is_treated from campaign_table, post_promo_visit as conversion proxy); observational treatment assignment. See disclaimer above.*

On the 2,500-customer grocery dataset the X-learner (n_estimators = 200) with real Dunnhumby campaign labels produces $3,207 in estimated Δprofit across 72 approved broadcast customers. The key finding is the Champions segment result: 63/506 Champions are approved based on genuine incremental response signal from the `campaign_table` data, while 443/506 Champions are suppressed as purchasing regardless. Potential Loyalists are suppressed at 99.5% (9/1,923 approved), correctly identifying that the vast majority of mid-tier Dunnhumby households purchase independently of campaign contact. A naive broadcast campaign on this

dataset produces estimated incremental profit of −$262 — worse than sending nothing at all. The pipeline (X-learner) produces $3,207 — a $3,469 improvement over naive broadcast (+1,325%). The CFO Guardrail scales in value with dataset size: on a larger population with more money-losing customers to block, its financial contribution grows proportionally.

Dataset 4 — UK Online Retail (4,338 customers, temporal-split proxy labels)

Method	Customers	Est. Incremental Profit†	Δ vs Full Pipeline
Full Pipeline — X-learner (n_est = 200)	19	**$2,529**	—
No Campaign (baseline)	—	$0	−$2,529
RFM-Only	N/A	N/A	N/A

†*Temporal-split proxy labels (converted = retention in final 25% of observation window; random treatment assignment). Prior figures of 63/$40,630 were from a now-corrected RFM proxy with feature leakage (was_treated = rfm_decile ≥ median; those same RFM features used as model inputs). Corrected proxy produces conservative honest estimates; comparison with broadcast-all requires a dedicated evaluation run with guardrail disabled.*

Scaling pattern: The pipeline's advantage compounds with dataset size as the proportion of near-zero-uplift customers grows. Dataset 1 (n=200) produces 0 broadcast under correctly calibrated randomized labels — a null result validating model calibration. Dataset 2 (n=905) $9,224; Dataset 3 Dunnhumby (n=2,500) +1,325% ($3,207 vs −$262 broadcast-all). Dataset 4 UK retail (n=4,338) with temporal-split proxy: $2,529 for 19 broadcast-approved customers. Under structured synthetic treatment labels (post-hoc, 82.5% broadcast, $55,324 Δprofit), the range of outcomes depending on label quality spans ~22×, confirming that label fidelity is the primary determinant of broadcast volume. Real-world validation with the forthcoming field study will establish causal estimates under randomized holdout conditions.

Note: The 'RFM-Only' baseline uses prob_control as the conversion estimate (near-zero for purely promo-driven customers), making promo costs dominate and producing negative incremental profit — confirming that uplift scoring is a prerequisite for the guardrail to operate selectively.

6.10.4 Ablation Study

Note: The 'No Uplift Model' row reflects a structural dependency: without uplift scores, the per-customer incremental profit $\Delta profit_i = 0 - promo_cost < 0$ for all customers, causing the CFO Guardrail to block all campaigns. The ΔB2,162 figure above is from the **T-learner baseline** row (prior proxy-label mode), not the X-learner full pipeline which produces B0 on correctly randomized labels. This distinction is important: the X-learner correctly produces 0 broadcast (validating null-treatment calibration), while the T-learner baseline under proxy labels produces B2,162 from 58 customers. The combined contribution figures should be read in context of their respective model/label rows.

Dataset 1 (Synthetic Restaurant, 200 customers):

Configuration Removed	Expected Profit	Δ vs Full	Interpretation
Full Pipeline (X-learner)	B0	—	Baseline (0 broadcast — null result validates calibration)
Full Pipeline (T-learner baseline)	B2,162		

Configuration Removed	Expected Profit	Δ vs Full	Interpretation
		N/A (different label mode)	T-learner with prior proxy labels
− CFO Guardrail	−฿533	**−฿533**	Guardrail prevents money-losing campaigns even at 0 broadcast
− Context Layer (Layer 3)	฿0	±฿0	No items flagged out-of-stock on this dataset — see Dataset 5 below
− Uplift Model (Layer 4)	฿0	±฿0	Without uplift, incremental profit = −promo_cost → guardrail blocks all (same result)
− Guardrail + Context	฿0	±฿0	Combined RFM-only: guardrail still fires without uplift

Note: With 0 broadcast on the full pipeline, the ablation Δprofit for most rows is ฿0. The meaningful ablation is removing the CFO Guardrail, which reveals that without financial constraint checking, −฿533 in money-losing campaigns would be dispatched (customers with negative expected incremental profit after promo cost). The guardrail's value on Dataset 1 is therefore ฿533 saved from the no-guardrail scenario.

Dataset 5 (Layer 3 Synthetic Test, 500 customers) — Layer 3 isolated contribution:

Configuration	Broadcast	Out-of-Stock Blocked	Interpretation
Full Pipeline (with Layer 3)	198	**23 customers blocked**	Layer 3 removes unfulfillable campaign targets
Without Layer 3 (in_stock = True for all)	221	0	23 customers incorrectly dispatched for stockout items
Layer 3 contribution	**−23**	**−23**	Directly prevents 23 customers from receiving promotions on out-of-stock products

Three findings stand out:

1. **The CFO Guardrail prevents money-losing campaigns on Dataset 1 even when the full pipeline produces 0 broadcast**, preventing ฿533 in negative expected profit from a no-guardrail scenario. The zero broadcast outcome itself validates the X-learner's calibration on randomized labels — no spurious uplift signal is manufactured. This strongly validates the architectural decision to embed financial constraint checking inside the ML decision loop.

2. **Context detection (Layer 3) shows no effect on Dataset 1**, as the velocity-based stock estimator flags no items as low-stock under the controlled restaurant generation process. This is a dataset limitation, not an architectural limitation.

3. **On Dataset 5 (engineered demand surges), Layer 3 directly contributes**: 23 customers (4.6%) are correctly suppressed from campaign dispatch because their primary product is out of stock. Without Layer 3, these customers would receive promotions for items unavailable for fulfillment — an error that would erode customer experience and negate the campaign's revenue goal. This validates Layer 3's architectural role in production deployments with real stock variability.

6.10.5 Multi-Seed Stability

The pipeline was run with 10 different LightGBM random seeds (0–9) to assess model stability:

Metric	Mean	SD	Stability
Total expected profit	฿2,162	±฿0	Fully deterministic
AUUC	0.1487	±0.0000	Fully deterministic
Guardrail blocked	142.0	±0.0	Fully deterministic

Note: The multi-seed AUUC of 0.1487 was computed on proxy labels (pre-correction). On randomized labels with the corrected label source, the AUUC is 0.0612 (T-learner; see Section 6.10.1). The 0.1487 value reflects proxy label bias and should not be used as a performance estimate.

Zero variance across seeds indicates deterministic convergence: LightGBM with n_estimators=30 on a 200-customer dataset finds the same splits regardless of random seed. Note that the production model now uses n_estimators=200 under the X-learner architecture; this multi-seed analysis was conducted on the T-learner baseline for reproducibility of the reported AUUC figure. This is an honest limitation of evaluating on a small synthetic dataset — it does not mean the model has zero variance in general, but rather that the dataset is too small to exhibit training-set sampling variance across seeds. This confirms the need for real pilot data to conduct statistically meaningful multi-seed evaluation. For the purpose of this paper, deterministic convergence ensures full reproducibility of all reported results.

6.10.6 R-Learner Results on X5 RetailHero

The R-learner (implemented as CausalForestDML from EconML) was evaluated on a 50,000-customer stratified sample drawn from the full 200,039-client X5 dataset to confirm that the pipeline produces a positive uplift signal on real, causally identified data. The 50K sample preserves the 50/50 treatment/control balance and the observed +3.32 pp population ATE.

Results table:

Model	Dataset	Features	AUUC (corrected)	Notes
R-Learner (CausalForestDML)	X5 (50K sample)	8 RFM	+0.0010	Positive signal confirmed
Random baseline	X5	—	-0.0017	
X-Learner	Dataset 1 (synthetic)	8	+0.0475	Null data, expected near-zero

The R-learner AUUC of +0.0010 is small in absolute terms but directionally correct and above the random baseline of −0.0017 — confirming that the pipeline extracts a genuine positive uplift signal from real randomized data. The magnitude is consistent with expectations: on a dataset where the population ATE is only +3.32 pp, heterogeneous treatment effects are modest, and an AUUC meaningfully above zero already indicates that the model is identifying genuinely more persuadable customers rather than ranking by purchase propensity alone.

AUUC formula methodological note: The standard Qini formula requires that the random baseline be normalized by the treatment/control ratio. When treatment rate ≠ 0.5, the naive formula (treating the random baseline as a straight diagonal line from origin to total converted) produces a floor of approximately −0.15 even for random models, because the unnormalized diagonal does not match the actual random targeting curve. The corrected formula used throughout this paper defines the random baseline as a straight line from 0 to the population ATE scaled by sample size:

random baseline endpoint=(∑iYiTiNT−∑iYi(1−Ti)NC)×N\text{random baseline endpoint} = \left(\frac{\sum_i Y_i T_i}{N_T} - \frac{\sum_i Y_i (1-T_i)}{N_C}\right) \times N

where NTN_T and NCN_C are treated and control counts respectively, and N=NT+NCN = N_T + N_C. This normalization ensures that a model with no discriminative power (random ordering) produces AUUC = 0 regardless of treatment rate, making AUUC values comparable across datasets with different treatment fractions. All AUUC figures reported in this paper use this corrected formula.

Heterogeneous treatment effect finding: Segment-level analysis of the R-learner's predicted individual treatment effects (ITEs) reveals meaningful heterogeneity consistent with the core business hypothesis:

> Low-frequency (lapsed) customers show +4.25% uplift vs +1.44% for high-frequency loyal customers — confirming the core business hypothesis that campaign resources should be concentrated on at-risk and occasional segments rather than champions.

This finding directly validates the pipeline's architectural decision to suppress Champions from campaign dispatch (Sections 6.1, 6.3, 6.4): high-frequency buyers respond to the SMS regardless of targeting, making campaign spend on this segment largely wasteful. The +4.25% uplift for lapsed customers represents the actionable incremental revenue the pipeline is designed to capture.

6.10.7 Hillstrom Cross-Domain Evaluation

The pipeline was evaluated on the MineThatData Hillstrom email campaign dataset (64,000 customers, randomised email A/B test) to validate performance on a fundamentally different domain and outcome structure. Unlike the X5 RetailHero dataset (which measures store visit conversion with +3.32 pp ATE), the Hillstrom data uses a binary email response outcome (visit or purchase within 2 weeks) where the observed ATE is approximately +6.2 pp (control visit rate 10.6%; mens email 18.3%; womens email 15.1%). This dataset tests whether the pipeline's heterogeneous treatment effect detection generalises beyond high-frequency retail transactions to lower-frequency, discrete campaign responses.

Dataset characteristics: The 64,000-customer cohort is divided into three groups: Womens Email (21,387), Mens Email (21,307), and No Email control (21,306), resulting in a 67/33 treatment/control imbalance. The actual observed ATE is approximately +6.2 pp (visit rates: control 10.6%, mens email 18.3%, womens email 15.1%). Eighteen engineered features are available (recency, history_log, gender flags, zip code indicators, channel indicators, and history-

segment one-hot encodings — see Section 4.4.10) after label-encoding of categorical variables. The imbalanced treatment rate makes HTE estimation more challenging than the 50/50 splits in previous evaluations, as the control group is smaller and more sensitive to sampling variance.

Model	Dataset	Features	AUUC	Notes
R-Learner (CausalForestDML)	Hillstrom (64K)	18 engineered	+0.0022	Positive signal confirmed
DR-Learner (LinearDRLearner)	Hillstrom	18 engineered	**+0.0032**	Best single-model performance
Ensemble (R+DR softmax)	Hillstrom	18 engineered	+0.0028	Near-equal weights (0.50/0.50)
Random baseline	Hillstrom	—	~0.0000	Corrected formula (67/33 split)

Note: The AUUC values above (0.0022, 0.0032, 0.0028) are from the SOTA models (CausalForestDML, DR-Learner) evaluated here. These differ from the pooled X-learner AUUC of 0.0614 reported in Section 6.11, which uses a different model architecture on the same dataset. The two sections measure different models on the same data — not a contradiction.

The DR-Learner achieves the highest AUUC at +0.0032, edging out the R-Learner (+0.0022). The ensemble using softmax-weighted combination of R-Learner and DR-Learner predictions yields +0.0028 with weights converging to near-equal (0.50/0.50), indicating that both base learners capture complementary heterogeneity patterns. The absolute AUUC values are small but directionally correct and above random. This magnitude is expected given the dataset characteristics: population ATE is approximately +6.2 pp, and the SOTA AUUC values here reflect the SOTA models' ability to rank customers by HTE on this dataset.

The 67/33 treatment/control imbalance introduces additional estimation difficulty versus balanced 50/50 designs. Despite this structural disadvantage, the pipeline detects a positive AUUC signal, validating that heterogeneous treatment effect detection generalises across domains, outcome types, and treatment rate configurations. The ensemble's near-equal learned weights suggest that on a low-signal cross-domain task, both forest-based (CausalForestDML) and linear (LinearDRLearner) approaches contribute equivalently to identifying persuadable segments. The Hillstrom ensemble model is saved to `models/registry/ensemble_sota_hillstrom.pkl` and represents the current cross-domain champion for email campaign uplift scoring.

6.11 Pooled Multi-Treatment Uplift — Hillstrom Dataset 6

The `PooledMultiTreatmentModel` (Section 4.4.10) was trained on the Hillstrom Dataset 6 (64,000 customers, 80/20 stratified train/holdout split, seed = 42). The pooled X-learner uses 20 features: 18 base customer features plus two arm indicators (`arm_mens_email`, `arm_womens_email`). Control observations have both arm features set to 0.

Training data composition:

Group	N	Visit rate
Control (no email)	17,045	10.7%
Treated — mens email	17,045	18.4%
Treated — womens email	17,110	15.2%
Treated (all)	34,155	16.8%

Holdout evaluation (12,800 customers, 20% stratified holdout):

Metric	Value
Qini coefficient	**0.1215**
AUUC	0.0614
ATE predicted	+0.0560
ATE actual	+0.0616
ATE calibration error	**9%**

The Qini coefficient of 0.1215 and AUUC of 0.0614 confirm that the pooled X-learner identifies a meaningful heterogeneous treatment effect across the population. The ATE calibration error of 9% on the Hillstrom RCT — substantially lower than the 247% error observed under proxy-label training on retail datasets, where the absence of true randomization inflates apparent ATE discrepancy — confirms that true RCT labels substantially improve model reliability. The improvement reflects both better label quality (true randomization vs. observational proxy) and the use of a different dataset; it is not a controlled comparison between the two training regimes on the same data.

Per-arm CATE distributions (holdout):

Arm	Avg CATE	Observed uplift	CATE > 0
mens_email	+0.0558	+0.0748	91.8%
womens_email	+0.0542	+0.0485	91.2%

Both arms produce positive average CATE, consistent with the population-level uplifts. The mens-email CATE exceeds the womens-email CATE on average, correctly reflecting the stronger empirical uplift (+7.7pp vs +4.5pp). The 91%+ fraction of customers with positive CATE reflects the strong baseline treatment effect across this population — a characteristic of email campaigns directed at pre-existing customers in a retailer with established brand affinity.

Best-arm assignment (predict_best_treatment):

Arm	Customers assigned	Share
mens_email	8,788	68.7%
womens_email	4,012	31.3%

The 68.7%/31.3% split is consistent with the stronger arm-level uplift for mens email. The model learns that customers with higher `mens` purchase history (product category feature) and certain recency/channel profiles are better matched to mens-email content, while womens-email is preferred for a substantial minority. This heterogeneity demonstrates that the pooled model captures arm-level treatment effect heterogeneity despite sharing a single set of model parameters.

---

7. Discussion

7.1 The Declarative Rule Engine as Architectural Progress

The most significant architectural change in this iteration is the redesign of Layer 4's decision logic from an imperative priority-ordered chain to a declarative two-step engine. The original design's monolithic if/elif block encoded both control logic (which customers qualify) and

financial validation (whether the action is profitable) in a single function, creating several maintenance liabilities: adding a new campaign type required modifying the control flow; testing required tracing through nested conditions; and the financial viability check was applied uniformly to all customers regardless of segment type.

The declarative engine resolves these issues by separating concerns. Business rules are expressed as data structures in `action_rules.py`, inspectable and modifiable without touching evaluation code. Financial validation is encoded in two purpose-built functions (`_viable` and `_viable_loyalty`) that reflect a deliberate policy distinction: persuadable customers are held to a stricter standard than rescue or sure-thing customers. The `matches()` function tests rules against customer context in a single pass, making the rule set auditable. This design is consistent with established software engineering principles for reducing technical debt in ML systems [16].

The practical effect is visible in the dessert-ml.csv send rate progression: the declarative engine with rescue logic and `_viable_loyalty` accounts for the largest single-upgrade jump (~10–15 percentage points), confirming that a significant fraction of the original suppressed population consisted of high-value customers blocked by an overly conservative uniform financial check.

7.2 The Two-Viability-Function Design as Financial Policy

The distinction between `_viable` and `_viable_loyalty` encodes a nuanced financial policy that the original single-check design could not express. For persuadable customers, the financial case for a campaign rests on the incremental revenue it generates above the customer's baseline conversion probability. For sure-thing and rescue customers, the financial case rests on a different calculation: the cost of losing the customer's future lifetime value versus the cost of the promotion.

This distinction matters empirically. Under the original design, a Can't Lose Them customer with mildly negative uplift (e.g., −0.06) and high baseline conversion (e.g., 0.65) would be blocked by the do_not_disturb guard regardless of their historical spend. Under the new design, the rescue tier override reclassifies this customer as persuadable with rescue=True, the `_viable_loyalty` check passes (baseline_profit − promo_cost > 0), and a VIP Win-Back action is dispatched. The promotion is financially sound on a lifetime value basis even though the per-interaction incremental profit is mildly negative.

7.3 The IBM 10-Segment Standard as Interpretability Bridge

Adopting IBM 10-segment names as cluster labels — via rank-percentile normalization — provides an interpretability bridge between the statistical clustering output and the operator's mental model. A business owner who understands intuitively that "Champions" need appreciation rather than discounts, and that "Can't Lose Them" customers are worth aggressive spend to retain, can read the Layer 5 executive report without needing to understand K-Means geometry or silhouette scores.

The rule-based flags complement the K-Means names by guaranteeing that rare-but-critical states always appear in the segment list. In a large dataset with a small proportion of truly new customers, K-Means may assign those customers to a "Promising" cluster because they are most similar in RFM space to low-frequency recent buyers; the New Customer flag overrides this classification based on a direct recency + frequency threshold, ensuring the welcome offer logic applies. This hybrid design — statistical clustering for the bulk of the population, deterministic rules for edge cases — is a practical pattern for production systems where certain customer states require guaranteed handling regardless of training data coverage.

7.4 The CFO Guardrail as Organizational Boundary Automation

The most significant novel contribution of this framework is **architectural**: the placement of a financial constraint validator inside the ML decision loop rather than as a downstream organizational review step.

Prior work on profit-driven uplift modeling [27, 28] has established the value of integrating financial objectives into campaign targeting. Gubela et al. [27] and Devriendt et al. [28] show that incorporating revenue and cost structure into the uplift optimization objective improves financial outcomes over pure response-probability targeting. Our CFO Guardrail extends this direction by implementing financial validation as a *hard constraint* — rather than a soft objective — enforced per customer at inference time, and exposing all cost parameters through a single operator-editable configuration file.

In conventional enterprise deployments, data science teams generate recommendations reviewed by finance or marketing operations before deployment. This creates latency (typically days to weeks in SME contexts where reviews are informal), organizational friction, and the risk that financially sound recommendations are delayed or discarded for non-quantitative reasons.

By parameterizing financial constraints — COGS rate, margin floor, discount rates — and applying them per customer at inference time, the system eliminates the review step for routine campaigns. Edge cases surface naturally through the `Do Nothing` classification rather than requiring human escalation. The `business_config.py` file serves simultaneously as the operator's financial policy document and the system's decision boundary, creating a direct and auditable link between business rules and campaign outputs.

The ablation result (Section 6.10.4) quantifies this contribution empirically: the full pipeline — including the CFO Guardrail — produces $3,207 in Dunnhumby incremental Δprofit by converting a −$262 broadcast-all scenario into a net-positive outcome, consistent with the Guardrail being a key mechanism preventing money-losing campaigns.

7.5 Dynamic Cluster Selection as SME Adaptation

The results across both datasets illustrate why hardcoding cluster count is inappropriate for SME deployment contexts. The restaurant dataset (200 customers, dense repeat-visit behavior) produces optimal $k = 2$ with silhouette = 0.679; the Marketing Campaign dataset (905 customers, aggregate-level data) produces $k = 3$ with silhouette = 0.306. A system that hardcoded $k = 3$ for the restaurant dataset would artificially split a naturally binary customer structure, while hardcoding $k = 2$ for the marketing dataset would collapse meaningful behavioral distinctions.

The dynamic selection procedure, selecting k by maximizing silhouette score over $k \in \{2,\ldots,10\}$, automatically scales granularity to data density. This is a key enabler of the minimal-input design principle: operators do not need to tune segmentation parameters.

7.6 Accessibility versus Accuracy Trade-offs

The framework makes deliberate simplifications to maintain accessibility. These trade-offs are transparent:

Inventory estimation from sales velocity introduces estimation error in the absence of live stock feeds — typical for SME POS systems. Operators with inventory management systems can replace the velocity-based estimator with direct stock queries through the `stock_restock_cycle_days` and `stock_buffer_multiplier` configuration parameters.

Treatment labels derived from the generator's randomized assignment in Dataset 1, and from observational campaign acceptance records in Dataset 2 — both requiring caution in causal interpretation as discussed in Sections 4.4.1 and 5.1 — is a necessary concession for SME settings where historical A/B data is unavailable. The 5% ongoing control group enables proper causal measurement going forward, converting the system from a batch estimator into an accumulating randomized experiment over time.

Local LLM inference (3B parameter model) produces adequate executive summaries at zero ongoing cost and with full data privacy, but may produce less nuanced analysis than larger hosted models. The structured output validation layer partially compensates by catching factual errors in segment names, customer counts, and profit figures.

7.7 Extensibility

The pipeline architecture is designed for vertical expansion. The six-column input schema is generic enough to accommodate any transactional business. The `business_config.py` central configuration enables parameter tuning without code changes. Planned extensions include:

- **Event-driven campaign prediction:** Incorporating scheduled events (festivals, public holidays, promotional calendars) as Layer 3 context signals for pre-emptive campaign planning
- **POS-CRM-LINE integration:** Connecting the pipeline output directly to LINE-based ordering and delivery systems, enabling automated campaign dispatch through existing customer communication channels
- **Multi-location aggregation:** Pooling POS streams from multiple outlets with site-level context flags for chain operators
- **Service business adaptation:** Mapping `product_name` to service SKUs for beauty, automotive, and professional services verticals

7.8 Budget Knapsack as Explicit Financial Constraint

The budget optimizer adds a second financial constraint layer operating at a different level than the CFO Guardrail. The Guardrail enforces per-customer margin and incremental profit floors — it eliminates individually unprofitable actions regardless of budget. The budget knapsack enforces a collective ceiling — it selects the highest-value subset once all individual decisions have been approved.

The two constraints are composable by design: the Guardrail runs first (eliminating loss-making customers), and the knapsack runs second (selecting the best subset of the survivors). This ordering matters: a naive knapsack over all customers (including unprofitable ones) would waste budget on customers who should not receive campaigns regardless of budget. The two-stage design ensures that budget is allocated only among customers where a campaign is financially warranted.

The diminishing marginal ROI property of the greedy selection is not merely a theoretical property — it encodes a practical business insight. The first ฿/$ 1,000 of campaign budget should go to customers where promo spend converts nearly 1:1 to incremental margin. The last ฿/$ 1,000 goes to borderline customers where the margin contribution is positive but thin. Operators who examine the bang-per-buck curve gain a direct read on whether their budget is appropriately sized for their customer base.

Implementation note: The budget knapsack works as designed but has not been evaluated against a real campaign with a fixed budget ceiling. Empirical validation is pending.

7.9 Pooled Learning vs One-vs-Rest for Multi-Treatment Uplift

The existing `MultiTreatmentUpliftModel` (one-vs-rest) fits separate X-learners for each treatment type, treating "received this treatment" vs "received anything else" as the binary treatment indicator. This design is statistically clean but inefficient: a customer who responds to discount offers and a customer who responds to loyalty points may share predictive features that one-vs-rest cannot exploit across models.

Pooled learning resolves this by encoding arm identity as part of the feature space. The tau (CATE) models learn a continuous function over (customer features, arm indicator), enabling the following signal-sharing patterns:

1. **Shared control outcome model (mu0):** Since mu0 predicts E[Y | X, T=0] without arm features, it pools all control observations across arms — yielding 3× the control sample size relative to per-arm fitting.
2. **Cross-arm CATE regularization:** The LightGBM regularization (reg_lambda, min_child_samples) in the tau models applies to the joint (X, arm) feature space, preventing overfitting to small-arm subpopulations.
3. **Arm-interaction features:** The model can discover interaction patterns, e.g., "customers with high `womens` purchase history respond better to womens email" — a cross-arm pattern not representable in separate one-vs-rest models.

The 9% ATE calibration error on the Hillstrom RCT (vs 247% under proxy-label training on retail datasets) is the most significant result in this evaluation. It is not primarily a property of pooled learning vs one-vs-rest — it is a consequence of using true randomized labels on a different dataset; the comparison is not controlled for dataset. However, the pooled model's Qini of 0.1215 on a balanced three-arm RCT provides an important reference point: this is the quality achievable when genuine randomization is available, establishing a performance ceiling that proxy-label models are expected to approach as real campaign response data accumulates.

7.10 Limitations and Future Work

Evaluation on synthetic data: Dataset 1 is synthetically generated from known segment parameters. The silhouette score (0.679) and k=2 selection reflect the generator's structure rather than discovering latent behavioral segments in real transaction data. Deterministic multi-seed convergence confirms the 200-customer dataset is too small for variance quantification. Evaluation on real pilot deployment data is the critical next step.

Uplift model AUUC on randomized labels: Apples-to-apples AUUC evaluation on Dataset 1's actual randomized `is_treated` labels yields T-learner = 0.0612 and X-learner = 0.0475 — both near-zero, the statistically correct result when no true treatment effect exists. The previously-reported T-learner AUUC of 0.2351 was computed on proxy labels (`was_treated = rfm_decile_segment ≥ median`), which correlate with training features by construction and inflate apparent discriminative ability. The corrected values are reported in Section 6.10.1. AUUC on causally identified holdout data from a live A/B deployment remains the most informative evaluation and will be reported in the forthcoming field study.

Resolved: proxy label feature leakage (v1.15). An earlier proxy label implementation derived `was_treated = rfm_decile ≥ median` and `converted = frequency > 1`, then used `rfm_decile_segment` and `frequency` as model input features — producing circular training. This was identified as a significant defect and corrected in v1.15: proxy labels now use a temporal-split design (converted = retention in final 25% of observation window; random treatment assignment), and proxy-mode features are restricted to basket-level signals that do not

participate in label construction. UK retail proxy broadcast count corrected from 63 to 19; Δprofit corrected from $40,630 to $2,529. See Section 4.4.1.

Resolved: uncalibrated base learner probabilities (v1.15). Raw LightGBM `predict_proba()` scores (ECE = 0.2496) were used directly in the profit formula, inflating or deflating incremental profit estimates proportionally. Fixed by wrapping μ_0 and μ_1 with `CalibratedClassifierCV(isotonic, cv=3)` (ECE = 0.0323, 87% reduction). See Section 4.4.1.

Resolved: LLM hard dependency (v1.15). The `import ollama` statement at module level caused `ModuleNotFoundError` on machines without Ollama installed, crashing the entire pipeline at import time. Fixed by lazy import with a structured template fallback. See Section 4.5.

Remaining open: A/B holdout feedback loop now closed (v1.15). The `holdout_evaluator.py` module now ingests post-campaign outcomes and exports retrain-ready labeled data. The remaining gap is that no real campaign has yet been deployed and measured — causal AUUC from a live holdout remains forthcoming.

Uplift evaluations in Datasets 1 and 2 rely on synthetic or no-control-group data: Dataset 1 uses treatment labels generated by a synthetic simulator — treatment effect is by construction null, and AUUC correctly converges to near-zero. Dataset 2 (`marketing_campaign.csv`) has no control arm: all 2,240 households were contacted (`Z_CostContact = 3` uniformly), making it structurally impossible to train or evaluate uplift from this source alone. Uplift estimates on Dataset 2 reflect customer acceptance propensity, not incremental treatment effect. The Dunnhumby real-data retraining methodology (Section 4.4.2) directly addresses this limitation by providing a genuine natural treatment/control split (1,584 treated / 916 control) and doubly-robust evaluation to correct for the observed 4.7× pre-campaign spend differential between groups.

Observational labels in Dataset 2: Marketing campaign acceptance (`AcceptedCmp`) reflects customer self-selection, not randomized assignment. Uplift estimates from Dataset 2 are observational and subject to confounding. They should not be interpreted as causal effects.

Baselines gap on small dataset: Dataset 1 (synthetic restaurant, 200 customers) produces 0 broadcast and Δprofit = ฿0 under the X-learner with correctly randomized labels — zero broadcast validates model calibration on a null treatment effect. The T-learner baseline produces ฿2,162 (+0.3% vs broadcast-all ฿2,109). The material differential appears in the comparison to RFM-only (−฿1,334 vs ฿2,162 T-learner), confirming that the guardrail and uplift model together prevent money-losing campaigns.

Context layer — validated on engineered dataset, pending real POS confirmation: The velocity-based stock estimator showed no effect on Datasets 1–4, all of which lacked sufficient demand variability to trigger stockout conditions. Dataset 5 (synthetic Layer 3 test) confirms that the architecture functions correctly when demand surges occur: all 3 engineered stockout products are correctly flagged `in_stock = False`, and 23 customers (4.6%) are blocked from campaign dispatch as a result (Section 6.5). The remaining open question — whether Layer 3 produces material suppression rates on real POS streams with genuine seasonal demand spikes — will be addressed in the production field study.

Association rule lift: Lift values in the synthetic dataset reflect structured menu co-purchase patterns by construction. Real-world deployment is expected to produce lower but more actionable lift values as order diversity increases.

BG/NBD convergence on high-frequency retail: The CLV rescue tier implementation (Section 4.4.9) exposes a known limitation of the BG/NBD survival model: the optimizer fails to converge

on daily/weekly purchasing datasets (dessert bakery, grocery retail) where customer dropout rates are near zero and transaction rates are very high. This is a fundamental BG/NBD characteristic, not a code defect — the model assumes a relatively low-frequency, non-contractual relationship with observable churn periods. For bakery and grocery contexts, an alternative survival specification (shifted-beta-geometric for contractual settings, or a simple recency-decay heuristic calibrated to observed churn rates) may provide more reliable CLV estimates. Until that alternative is implemented, `_viable_clv` falls back to `_viable_loyalty`, preserving current behavior.

Future work will examine: (1) real pilot deployment data from partner restaurant operators with 5% control groups; (2) online learning to update RFM and uplift models continuously; (3) multi-armed bandit formulations for campaign action selection with exploration-exploitation tradeoffs; (4) Qini-curve-based uplift objective integration to replace the current net-margin floor with a continuous financial objective in the LightGBM training loss; (5) alternative CLV survival models (Pareto/NBD, recency-decay heuristics) suited to high-frequency retail; (6) integration of pooled multi-treatment CATE estimates into the declarative rule engine's action selection, enabling per-arm offer type routing based on model-predicted best treatment.

7.11 Expected Production Performance

The gap between model-estimated performance and realized production outcomes is a critical consideration for practitioners evaluating whether to deploy the framework. This section provides an honest, structured forecast grounded in the known limitations of each pipeline component.

What is expected to transfer reliably to production:

Segmentation quality — Layer 1 silhouette scores are computed on actual transaction data and reflect genuine cluster structure in the input distribution. Segment assignments will be consistent with the patterns observed across datasets. The dynamic k selection has demonstrated stable behavior across four structurally different datasets ($k \in \{2, 3\}$) and is unlikely to degrade on a new POS export from the same business vertical.

CFO Guardrail direction — The fundamental logic is sound regardless of label quality: customers who purchase at high baseline frequency generate near-zero incremental revenue from a promotion while incurring its full cost. The guardrail's function of blocking these customers is expected to hold in production. The scaling pattern observed across datasets (guardrail savings grow with customer base size) is consistent with this mechanism.

broadcast_list.csv accuracy — With a real CRM providing verified customer IDs, the join between pipeline decisions and the customer database will be exact, eliminating the proxy-matching approximations present in the public dataset evaluations.

What will likely transfer with attenuation:

Uplift model rankings — The current model is trained on observational or proxy labels, producing rankings that reflect purchase propensity rather than causally identified treatment response. In production, the model will continue to rank customers in the correct directional order — high-uplift customers will tend to respond more to campaigns than low-uplift customers — but the magnitude of the estimated uplift scores will likely be inflated relative to the true causal effect. Empirical evidence from the uplift modeling literature [12] suggests that observational uplift estimates typically overstate realized incremental lift by a factor of 1.5–3×.

Estimated incremental profit figures — The model-estimated improvements reported in this paper (+1,325% on Dunnhumby, +3,354% on UK retail — see Section 6.6) are *(model-estimated, pre-correction)* projections and should be treated as upper bounds rather than production

expectations. These figures are based on observational or proxy labels before the corrected temporal-split methodology was fully applied. A realistic range for the first deployment cycle, based on industry benchmarks for campaign personalization at comparable scales, is **+20–50% improvement in campaign ROI vs. naive broadcast**. This range accounts for: (a) attenuation from observational label bias; (b) real-world behavioral variance not captured by the model; and (c) novelty effects that inflate early campaign response rates before settling to steady state.

What requires real A/B data to determine:

Question	Observable After
Does the guardrail prevent realized losses?	1 campaign cycle (4–8 weeks)
Does uplift ranking identify real responders?	2–3 campaign cycles
What is the true AUUC on causal labels?	3–6 months (with 5% control group)
At what n does guardrail value exceed deployment cost?	6 months cross-site comparison

Recommended production monitoring metrics:

- **Incremental Revenue Lift**: revenue of Treatment group minus revenue of Control group, normalized per customer — the primary causal KPI
- **Campaign ROI**: (incremental revenue − promo cost) / promo cost — financial efficiency metric
- **Guardrail Precision**: fraction of guardrail-blocked customers who would actually have generated negative incremental profit — validates suppression accuracy
- **Uplift Rank Correlation**: Spearman correlation between predicted uplift rank and realized conversion rate by decile — tracks whether the model's ranking is causally predictive

The forthcoming MVP field study with the CRM-integrated POS partner — incorporating a 20% randomized holdout control group and per-customer revenue attribution — will provide the first causally identified estimates for all four metrics above.

---

8. Conclusion

This paper presented the ML Campaign Engine, a five-layer MLOps pipeline that enables small and medium enterprises in retail and food service to generate financially validated, per-customer campaign recommendations from standard POS transaction exports. The system requires no dedicated data science expertise, no additional data infrastructure beyond an existing POS system, and produces outputs in plain-language executive summary format suitable for direct operator consumption.

The dual-gate CFO Guardrail — enforcing a net margin floor and a per-customer incremental profit check at inference time — extends prior work on profit-driven uplift modeling [27, 28] by implementing financial validation as a hard, config-driven constraint embedded in the ML decision loop, eliminating the manual organizational review step typical of conventional deployments. Ablation analysis demonstrates that the CFO Guardrail is the primary mechanism preventing money-losing campaigns, producing $3,207 in Dunnhumby incremental Δprofit by converting a −$262 broadcast-all scenario into a net-positive outcome.

The uplift model has been upgraded from the T-learner baseline (prior reported AUUC = 0.2351 on Dataset 1 was inflated by proxy label bias; corrected value on randomized labels = 0.0612; X-learner AUUC on randomized labels = 0.0475) to an X-learner meta-learner (Künzel et al. 2019, n_estimators = 200), which trains five LightGBM models through a two-stage imputation procedure that better handles the treatment/control group imbalance typical of SME deployments. Additionally, a purpose-built Layer 3 synthetic test dataset (Dataset 5, 500 customers) validates the context detection layer's stockout suppression: all three engineered demand-surge products are correctly flagged out-of-stock, and 23 customers (4.6%) are blocked from campaign dispatch — confirming that the previously-reported zero Layer 3 effect was a dataset limitation and not an architectural one.

Evaluation across six datasets confirms context-appropriate behavior and a consistent scaling pattern. The synthetic restaurant dataset (200 customers) yields k = 2 with silhouette = 0.679, T-learner AUUC = 0.0612 (corrected from prior inflated value of 0.2351; see Section 6.10.1), and pipeline marginally outperforming naive broadcast (+0.3%) under randomized labels. The Dunnhumby grocery dataset (2,500 customers) yields k = 3 with silhouette = 0.595. With the X-learner uplift model, the full pipeline outperforms broadcast-all by +1,325% in model-estimated incremental profit ($3,207 vs −$262.05, 72/2,500 broadcast). Notably, X-learner approves 63/506 Champions based on genuine incremental response signal from real campaign data, while suppressing the remaining 443/506 Champions as purchasing regardless of promotions — concentrating broadcast on those with genuine incremental response. The UK Online Retail dataset (4,338 customers) yields k = 2 with silhouette = 0.931. Under temporal-split proxy labels (corrected from RFM proxy with feature leakage), the X-learner pipeline approves 19 customers for broadcast (Δprofit = $2,529). Under synthetic treatment labels (post-hoc), 3,581 customers are approved (Δprofit = $55,324) — the ~188× range validates that label fidelity is the primary driver of broadcast volume rather than architectural conservatism. Dynamic cluster selection correctly adapts across all data structures. All six stress-test scenarios pass quantitative criteria. The scaling pattern — pipeline advantage grows from +0.3% at n=200 (T-learner baseline; the X-learner produces 0 broadcast on correctly randomized labels, validating calibration) to +1,325% at n=2,500 — is consistent with the hypothesis that guardrail value compounds as the fraction of near-zero-uplift customers increases. All profit figures are model-estimated projections; causal validation requires the forthcoming real-world field study.

Honest limitations include the small synthetic dataset, deterministic model convergence at n=200, and the absence of real pilot data. All profit figures reported in this paper are model-estimated projections on synthetic or public datasets — they represent the system's designed behavior under controlled conditions, not realized revenue outcomes from live campaigns.

Research Roadmap: Real-world profit impact will be measured in a forthcoming field study conducted with a CRM-integrated POS operator, incorporating randomized holdout control groups (Treatment / Control split) and realized profit attribution across full campaign cycles. That study will provide the first causally interpretable evidence of whether the pipeline's financially validated recommendations produce measurable incremental revenue above the no-campaign baseline.

Three extensions further the framework's financial rigor and modeling capability. A **budget-constrained knapsack optimizer** (`budget_optimizer.py`) selects the profit-maximizing subset of CFO-approved customers within an operator-specified spend ceiling, using greedy bang-per-buck ordering that is provably optimal for the fractional knapsack and achieves a **1/2-approximation** for the 0/1 knapsack via *max(greedy result, best single item)* ≥ ½ × OPT *(implementation complete; empirical validation against a real fixed-budget campaign is pending)*. A **CLV-informed rescue tier viability function** (`_viable_clv`) conditions rescue campaign approval on `promo_cost ≤ clv_at_risk` estimated from BG/NBD survival modeling — replacing the heuristic `_viable_loyalty` check with a financially grounded justification; fallback to the heuristic is automatic when BG/NBD fails to converge on high-

frequency datasets *(BG/NBD falls back to heuristic on all tested datasets; empirical CLV results are pending)*. Finally, a **pooled multi-treatment uplift model** (`PooledMultiTreatmentModel`) trains a single X-learner with treatment arm as one-hot features, evaluated on the Hillstrom MineThatData RCT: Qini = 0.1215, AUUC = 0.0614, ATE calibration error of 9% on the Hillstrom RCT — substantially lower than the 247% error observed under proxy-label training on retail datasets, where the absence of true randomization inflates apparent ATE discrepancy. This confirms that true RCT data substantially improves model reliability, though the comparison reflects both better labels and a different dataset.

The framework establishes a reproducible, extensible baseline for democratizing campaign intelligence for SMEs, and provides a validated technical foundation for the real-world A/B deployment that will constitute the critical next phase of evaluation.

References

Customer Segmentation & RFM

[1] Hughes, A. M. (1994). *Strategic Database Marketing*. Probus Publishing Company, Chicago.

[2] Bult, J. R., & Wansbeek, T. (1995). Optimal selection for direct mail. *Marketing Science*, 14(4), 378–394. https://doi.org/10.1287/mksc.14.4.378

[3] Chen, D., Sain, S. L., & Guo, K. (2012). Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining. *Journal of Database Marketing & Customer Strategy Management*, 19(3), 197–208. https://doi.org/10.1057/dbm.2012.17

[4] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297. University of California Press.

[5] Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

[6] Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

Market Basket Analysis

[7] Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94)*, pp. 487–499. Morgan Kaufmann.

[8] Hahsler, M., Grün, B., & Hornik, K. (2005). arules — A computational environment for mining association rules and frequent item sets. *Journal of Statistical Software*, 14(15), 1–25. https://doi.org/10.18637/jss.v014.i15

[9] Kaur, M., & Kang, S. (2016). Market basket analysis: Identify the changing trends of market data using association rule mining. *Procedia Computer Science*, 85, 78–85. https://doi.org/10.1016/j.procs.2016.05.180

Uplift Modeling

[10] Radcliffe, N. J., & Surry, P. D. (1999). Differential response analysis: Modelling true responses by isolating the effect of a single action. In *Proceedings of Credit Scoring and Credit Control VI*. University of Edinburgh.

[11] Gutierrez, P., & Gérardy, J. Y. (2016). Causal inference and uplift modelling: A review of the literature. In *JMLR Workshop and Conference Proceedings: ICML 2016 Workshop on Predictive Modeling*.

[12] Devriendt, F., Moldovan, D., & Verbeke, W. (2018). A literature survey and experimental evaluation of the state-of-the-art in uplift modeling. *Big Data*, 6(1), 13–41. https://doi.org/10.1089/big.2017.0104

[13] Rzepakowski, P., & Jaroszewicz, S. (2012). Decision trees for uplift modeling with single and multiple treatments. *Knowledge and Information Systems*, 32(2), 303–327. https://doi.org/10.1007/s10115-011-0434-0

Gradient Boosted Trees

[14] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30, pp. 3146–3154.

[15] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference*, pp. 785–794. https://doi.org/10.1145/2939672.2939785

MLOps & Technical Debt

[16] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J. F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS 2015)*, 28, pp. 2503–2511.

[17] Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11, 31866–31879. https://doi.org/10.1109/ACCESS.2023.3262138

[18] Paleyes, A., Urma, R. G., & Lawrence, N. D. (2022). Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6), 1–29. https://doi.org/10.1145/3533378

Champion-Challenger & Model Governance

[19] Shmueli, G., Bruce, P. C., Gedeck, P., & Patel, N. R. (2020). *Data Mining for Business Analytics: Concepts, Techniques and Applications in Python*. John Wiley & Sons.

[20] Barocas, S., Hardt, M., & Narayanan, A. (2023). *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press.

LLM Integration & Local Inference

[21] Meta AI. (2024). *Llama 3.2: Multimodal large language models*. Meta AI Research Blog. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-foundation-model

[22] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., et al. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*.

[23] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38. https://doi.org/10.1145/3571730

SME Marketing & Technology Adoption

[24] Harrigan, P., Soutar, G., Choudhury, M. M., & Lowe, M. (2015). Modelling CRM in a social media age. *Australasian Marketing Journal*, 23(1), 27–37. https://doi.org/10.1016/j.ausmj.2014.11.001

[25] Nguyen, T. H., Newby, M., & Macaulay, M. J. (2015). Information technology adoption in small business: Confirmation of a proposed framework. *Journal of Small Business Management*, 53(1), 207–227. https://doi.org/10.1111/jsbm.12058

Dataset

[26] Saldanha, R. (2018). *Marketing Campaign Dataset*. Kaggle. https://www.kaggle.com/datasets/rodsaldanha/marketing-campaign

[29] Dunnhumby. (2020). *The Complete Journey*. Kaggle. https://www.kaggle.com/datasets/frtgnn/dunnhumby-the-complete-journey

Meta-Learners for Heterogeneous Treatment Effects

[30] Künzel, S. R., Sekhon, J. S., Bickel, P. J., & Yu, B. (2019). Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the National Academy of Sciences*, 116(10), 4156–4165. https://doi.org/10.1073/pnas.1804597116

[31] Dudík, M., Langford, J., & Li, L. (2011). Doubly robust policy evaluation and learning. In *Proceedings of the 28th International Conference on Machine Learning (ICML 2011)*, pp. 1097–1104.

[32] Athey, S., & Imbens, G. W. (2016). Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences*, 113(27), 7353–7360. https://doi.org/10.1073/pnas.1510489113

Profit-Driven Uplift Modeling

[27] Gubela, R. M., Lessmann, S., & Jaroszewicz, S. (2020). Response transformation and profit decomposition for revenue uplift modeling. *European Journal of Operational Research*, 283(2), 647–661. https://doi.org/10.1016/j.ejor.2019.11.030

[28] Devriendt, F., Berrevoets, J., & Verbeke, W. (2021). Why you should stop predicting customer churn and start using uplift models. *Information Sciences*, 548, 497–515. https://doi.org/10.1016/j.ins.2019.07.040

Multi-Treatment RCT Dataset

[33] Hillstrom, K. (2008). *MineThatData Email Analytics Challenge Dataset*. MineThatData Blog. https://blog.minethatdata.com/2008/03/minethatdata-e-mail-analytics-and-data.html

Probability Calibration

[35] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML 2005)*, pp. 625–632. https://doi.org/10.1145/1102351.1102430

Customer Lifetime Value & Survival Modeling

[34] Fader, P. S., Hardie, B. G. S., & Lee, K. L. (2005). “Counting your customers” the easy way: An alternative to the Pareto/NBD model. *Marketing Science*, 24(2), 275–284. https://doi.org/10.1287/mksc.1040.0098

Appendix A — Pipeline Configuration Parameters

All business parameters are centralized in `src/business_config.py`. Operators adjust this file to configure the pipeline for their cost structure without modifying pipeline code.

Parameter	Default	Description
`cogs_rate`	0.50	Cost of goods as fraction of revenue. Overridden by data-driven estimate (P10/mean per product) when ≥10 products have ≥5 observations
`min_allowable_net_margin`	0.15	CFO floor — campaigns below this margin → Do Nothing
`control_group_rate`	0.05	A/B holdout fraction reserved for causal measurement
`gate_growth_target_pct`	3.0	Challenger promotion threshold (%)
`recession_threshold`	0.65	Spend ratio below which recession mode activates
`peak_hour_start`	17	Fallback default only — overridden by data-driven detection (mean + 0.5×std of hourly transaction volume)
`peak_hour_end`	21	Fallback default only — overridden by data-driven detection (mean + 0.5×std of hourly transaction volume)
`business_tz`	Asia/Bangkok	Business timezone for peak detection
`off_peak_traffic_rate`	0.22	Promo cost rate for Off-Peak customers (aggressive traffic driver — heavier discount justified when store is quiet)

Parameter	Default	Description
`peak_upsell_rate`	0.08	Promo cost rate for Peak customers (light-touch upsell when customers already arriving)
`low_basket_threshold`	100	AOV below this → Do Nothing (Low Basket Size). Automatically replaced by P10 of actual `avg_spent` when dataset median `avg_spent` < this value (e.g. grocery datasets)
`high_uplift_threshold`	0.08	Uplift above this → bundle promotion
`vip_rfm_threshold`	8	RFM decile ≥ this → VIP reward
`stock_buffer_multiplier`	1.5	Estimated stock covers this multiple of avg demand per restock cycle
`stock_restock_cycle_days`	14	Assumed replenishment cycle in days
`low_stock_threshold`	5	Remaining stock ≤ this → exclude from bundle

Appendix B — Stress Test Scenario Definitions

Scenario	Volume	AOV Distribution	Temporal Pattern	Primary Test
`mega_sale`	3× normal	Right-skewed, high AOV	Peak-concentrated	Spike detection, guardrail under inflated margins
`recession`	0.6× normal	Left-shifted, low AOV	Uniform	Recession mode activation, Do Nothing rate
`bulk_shift`	Normal	Extreme right tail	Uniform	IQR outlier isolation
`cold_start`	0.2× normal	Normal	Uniform, short window	k = 2 fallback, sparse data handling
`off_peak`	Normal	Normal	00:00–16:00 concentrated	Off-peak context detection
`data_noise`	Normal	Normal + NaN injection	10% corrupted timestamps	Error handling, graceful degradation

Appendix C — Declarative Rule Engine Design Summary

The declarative rule engine (`src/action_rules.py`) defines the full action menu as two Python lists of dicts. The evaluation logic in `src/uplift_engine.py` calls `matches()`

and either `_viable` or `_viable_loyalty` for each rule without knowledge of the specific rule contents. This separation enables:

1. **Adding a new campaign type** by appending a dict to `SURE_THING_RULES` or `PERSUADABLE_RULES` with the appropriate tier, context, rfm_min, and rate_key fields — no changes to evaluation code required.

2. **Auditing the rule set** by reading the list data structure directly — no control flow tracing required.

3. **Testing individual rules** by constructing a mock customer context and calling `matches()` on a single rule dict.

The `cluster_tier()` function maps IBM segment names (including rule-based flag names) to the tier strings used in rule `tiers` lists. It checks flag-name prefixes first (priority overrides: `can't lose`, `churn risk`, `new customer`, `about to sleep`) before IBM K-Means labels, ensuring suffixed duplicates like "Champions (2)" map correctly to "champions" via `startswith` prefix matching.

The evaluation strategies are:

Rule pool	Strategy	Viability function
`SURE_THING_RULES`	First viable rule wins (list order)	`_viable_loyalty`
`PERSUADABLE_RULES`	Collect all viable, return max profit	`_viable` (or `_viable_loyalty` if rescue=True)

The rescue flag is set to True by `decide_for_customers()` when the customer's tier is in {churn_risk, cant_lose, hibernating, about_to_sleep} and their uplift falls in the do_not_disturb range but above the rescue_uplift_guard threshold (−0.10). The flag is passed into `build_action_menu()`, which selects the viability function accordingly.

The two viability functions implement the CFO Guardrail at the per-rule level:

- `_viable(rate_key)`: requires net_margin ≥ MIN_MARGIN AND incremental > 0. Used for persuadable customers where the campaign must generate more revenue than it costs at the margin.
- `_viable_loyalty(rate_key)`: requires baseline_profit − promo_cost > 0. Used for sure-thing and rescue customers where the financial case is lifetime value preservation, not per-interaction incrementality.

This two-function design means the guardrail applies appropriate standards to each customer type rather than a one-size-fits-all check, enabling the pipeline to serve the full range of IBM customer archetypes while maintaining financial discipline.

End of Draft v2.0 Classification: Confidential — Proprietary Research © 2026 Codera Solutions · Pennoung H. · All rights reserved